



Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería de Computadores

Curso 2025-2026

Trabajo Fin de Grado

**TELEOPERACIÓN DEL ROBOT TURTLEBOT
MEDIANTE UNA INTERFAZ DE REALIDAD
VIRTUAL**

Autor: Rodrigo Hervada Llahosa

Tutor: Jonathan Crespo Herrero

Agradecimientos

Agradezco a todas las personas que han contribuido, por acción o por omisión, a la realización de este trabajo.

Resumen

Este trabajo presenta el diseño e implementación de un sistema de telepresencia robótica inmersiva basado en el robot móvil Turtlebot2 y una interfaz de realidad virtual para las gafas Meta Quest 2. El sistema permite a un operador controlar en remoto el robot desde un entorno virtual, recibir retransmisión de vídeo en tiempo real desde la cámara RGB-D del robot, y mantener comunicación audiovisual bidireccional con interlocutores presentes en el entorno físico del robot mediante un dispositivo móvil Android.

La arquitectura del sistema integra tres componentes principales: el stack ROS Kinetic ejecutado sobre el Turtlebot2 con Ubuntu 16.04, una aplicación de realidad virtual desarrollada en Unity 2022 que actúa como núcleo del sistema, y una aplicación Android desarrollada en Android Studio que gestiona la comunicación multimedia con el usuario remoto. La comunicación entre Unity y ROS se realiza mediante el protocolo ROSBridge a través de WebSocket usando el paquete ROS#. Por otra parte, la transmisión de audio y vídeo se realiza sobre UDP para minimizar la latencia. El sistema incluye además una interfaz de usuario con información del sistema y el robot, como la batería y las velocidades, controles de acciones rápidas sobre la odometría y el SLAM, y un minimapa, entre otros. Además incluye control de teleoperación mediante VR y representación del operador mediante un avatar virtual con sincronización labial.

El proyecto demuestra que es posible implementar un sistema de telepresencia inmersiva funcional sobre hardware de bajo coste y plataformas con software legacy, utilizando exclusivamente herramientas de acceso libre.

Abstract

This project presents the design and implementation of an immersive robotic telepresence system based on the Turtlebot2 mobile robot and a virtual reality interface for the Meta Quest 2. The system allows an operator to remotely control the robot from a virtual environment, receive real-time video streaming from the robot's RGB-D camera, and maintain bidirectional audiovisual communication with interlocutors present in the robot's physical environment through an Android mobile device.

The system architecture integrates three main components: the ROS Kinetic stack running on the Turtlebot2 with Ubuntu 16.04, a virtual reality application developed in Unity 2022 that acts as the core of the system, and an Android application developed in Android Studio that manages multimedia communication with the remote user. Communication between Unity and ROS is carried out through the ROSBridge protocol via WebSocket using the ROS# package. On the other hand, audio and video transmission is performed over UDP to minimize latency. The system also includes a user interface with system and robot information, such as battery and speeds, quick action controls over odometry and SLAM, and a minimap, among others. Additionally, it includes teleoperation control via VR and operator representation through a virtual avatar with lip synchronization.

The project demonstrates that it is possible to implement a functional immersive telepresence system on low-cost hardware and legacy software platforms, using exclusively open access tools.

Acrónimos y abreviaturas

- **ROS**: Robot Operating System (Sistema Operativo de Robots)
- **VR**: Virtual Reality (Realidad Virtual)
- **ROS#**: ROS Sharp (Paquete para la comunicación entre Unity y ROS)
- **Rosbridge**: Rosbridge Protocol (Protocolo para la comunicación entre ROS y otras aplicaciones)
- **API**: Application Programming Interface (Interfaz de Programación de Aplicaciones)
- **SLAM**: Simultaneous Localization and Mapping (Localización y Mapeo Simultáneos)
- **HMD**: Head-Mounted Display (Pantalla Montada en la Cabeza)
- **RGB-D**: Red Green Blue - Depth (Sensor de color y profundidad)
- **FPS**: Frames Per Second (Fotogramas por segundo)
- **TCP**: Transmission Control Protocol (Protocolo de Control de Transmisión)
- **UDP**: User Datagram Protocol (Protocolo de Datagramas de Usuario)
- **PCM**: Pulse Code Modulation (Modulación por Impulsos Codificados)
- **JPEG**: Joint Photographic Experts Group (Formato de compresión de imagen)
- **URDF**: Unified Robot Description Format (Formato de Descripción de Robot)

Índice de contenidos

Índice de tablas	IX
Índice de figuras	XI
1. Introducción	1
1.1. Motivación	1
1.2. Objetivo general	2
1.2.1. Objetivos específicos	2
1.3. Estructura del documento	3
1.4. Metodología y plan de trabajo	4
1.4.1. Cronograma	6
1.4.2. Explicación por tarea	6
2. Estado del arte	9
2.1. Teleoperación, telepresencia y telerrobótica	9
2.2. Realidad virtual en robótica y teleoperación	10
2.3. Integración de ROS-Unity e infraestructuras legacy y modernas	13
2.4. Plataformas de telepresencia inmersiva con robot móvil	16
2.5. Comunicaciones multimedia en tiempo real	17
2.6. Integración de dispositivos móviles en sistemas de telepresencia	18
2.7. Comparativa y posicionamiento del proyecto	19
3. Diseño	21
3.1. Diseño del sistema	21
3.2. Criterios de diseño	22
3.3. Decisiones arquitectónicas	23
3.3.1. Topología centralizada	23
3.3.2. Separación por tipo de tráfico	23
3.3.3. Desacoplamiento del subsistema móvil	23
3.4. Flujo de interacción	23
3.5. Presupuesto y costes	24
4. Desarrollo	26

4.1.	Plataforma de desarrollo	26
4.1.1.	Hardware	26
4.1.2.	Software	28
4.1.3.	Herramientas adicionales	29
4.2.	Arquitectura general del sistema	29
4.2.1.	Decisiones de diseño transversales	30
4.3.	Entorno ROS en el Turtlebot2	31
4.3.1.	Stack de software en el robot	31
4.3.2.	Nodo de streaming de cámara RGB-D <code>udp_camera_stream_robust.py</code>	32
4.3.3.	Nodo de gestión del robot <code>Turtlebot2_robot_manager.py</code>	33
4.4.	Entorno de realidad virtual en Unity	33
4.4.1.	Realidad virtual e interfaz de usuario	33
4.4.2.	Comunicación con ROS y teleoperación	35
4.4.3.	Sistema multimedia bidireccional	35
4.5.	Aplicación Android	38
4.5.1.	Recepción del vídeo del avatar	39
4.5.2.	Recepción del audio del operador	39
4.5.3.	Envío del micrófono al operador	39
4.5.4.	Envío de la cámara al operador	39
4.5.5.	Interfaz de configuración	39
4.6.	Entorno simulado	40
4.6.1.	Configuración de la máquina virtual	40
4.7.	Problemas encontrados durante el desarrollo	41
4.7.1.	Obsolescencia de la plataforma móvil.	41
4.7.2.	Acceso limitado al robot real.	42
4.7.3.	Condiciones de red en el entorno real.	42
4.7.4.	Acumulación de latencia en los búferes de datos multimedia.	42
4.7.5.	Error en el paquete RosSharp.	42
4.7.6.	Funcionalidades descartadas.	43
4.7.7.	Sincronización entre hilos de ejecución.	44
5.	Experimentos y métricas	45
5.1.	Metodología experimental	45
5.2.	Parámetros de configuración del sistema	46
5.3.	Resultados	47
5.3.1.	Latencia	47
5.3.2.	Ancho de banda	47
5.3.3.	Tasa de refresco	48
5.3.4.	Calidad audiovisual	48
5.3.5.	Tareas de navegación	48
6.	Conclusiones y trabajos futuros	49

6.1. Discusión de resultados	49
6.2. Cumplimiento de objetivos	50
6.2.1. Objetivo general	50
6.2.2. Objetivos específicos	50
6.3. Conclusiones generales	51
6.4. Trabajos futuros y posibles mejoras	52
Bibliografía	53

Índice de tablas

3.1. Desglose de recursos materiales, uso, valor comercial y coste real de adquisición.	25
3.2. Desglose de los costes de personal.	25
5.1. Parámetros de configuración del sistema según entorno de prueba.	47
5.2. Latencia de red medida en ambos entornos de prueba.	47
5.3. Consumo de ancho de banda en la interfaz WiFi del Turtlebot según el entorno.	48

Índice de figuras

2.1.	*	12
2.2.	*	12
2.3.	Sistema NimbRo Avatar (figura extraída de [20]).	12
2.4.	*	12
2.5.	*	12
2.6.	Sistema BUT Telepresence (figura extraída de [21]).	12
2.7.	*	15
2.8.	Sistema ROS Reality (figura extraída de [26]).	15
2.9.	*	17
2.10.	Sistema Reality Fusion (figura extraída de [12]).	17
3.1.	Diagrama de la arquitectura del sistema.	22
4.1.	Montaje final del Turtlebot 2 con el dispositivo móvil Android que actúa como nodo de telepresencia.	27
4.2.	*	32
4.3.	*	32
4.4.	Capturas del entorno virtual mostrando los flujos RGB y de profundidad (Depth) generados por la cámara del robot.	32
4.5.	*	34
4.6.	*	34
4.7.	Paneles de la interfaz de usuario en Unity, mostrando el minimapa, estado de la batería, conexión de ROS y controles rápidos.	34
4.8.	*	36
4.9.	*	36
4.10.	Perspectiva del operador en realidad virtual recibiendo los flujos de vídeo del robot (pantalla principal) y del dispositivo móvil (pantalla secundaria).	36
4.11.	*	38
4.12.	*	38
4.13.	Aplicación Android actuando como nodo secundario, mostrando el avatar virtual del operador y los ajustes de conexión.	38

4.14. Vista cenital del entorno de pruebas simulado (<i>turtlebot_world</i>) en Gazebo.	40
5.1. *	46
5.2. *	46
5.3. *	46
5.4. Ambos entornos de pruebas, el simulado en Gazebo y el real en el laboratorio	46

1

Introducción

En este capítulo se presenta el contexto general del trabajo y se exponen los aspectos que motivan su desarrollo. Asimismo, se describen el objetivo general y los objetivos específicos planteados, la estructura seguida en el documento y la metodología junto con el plan de trabajo empleados para la realización del proyecto.

1.1. Motivación

En este proyecto se presenta un sistema de teleoperación del robot Turtlebot2 mediante una interfaz de realidad virtual que permite al operador controlar el robot desde una ubicación remota. En la actualidad, las reuniones y colaboraciones a distancia se realizan principalmente a través de videollamadas, formato que presenta limitaciones inherentes en términos de inmersión y presencia del usuario. Con el desarrollo de dispositivos de realidad virtual comerciales, se abre la posibilidad de mejorar significativamente esta experiencia, proporcionando al operador una sensación de presencia física en el espacio remoto de forma accesible.

El uso de gafas de realidad virtual para este fin ofrece ventajas claras sobre métodos tradicionales, ya que el operador no solo ve el espacio remoto, sino que se encuentra inmerso en él, lo que facilita una mejor comprensión del entorno y una interacción más natural. Además, al contar con un cuerpo físico, el operador dispone de capacidades que van más allá de la mera observación, pudiéndose desplazar libremente por el entorno, explorar diferentes espacios, seguir a otros interlocutores o posicionarse según conveniencia para mejorar la comunicación. To-

das estas posibilidades convierten reuniones remotas en experiencias más cercanas a encuentros presenciales.

El Turtlebot2 constituye una plataforma idónea para esta implementación. Se trata de un robot de bajo coste que, aunque descatalogado, sigue siendo funcional. Su disponibilidad como plataforma de desarrollo, junto con la asequibilidad de dispositivos de realidad virtual modernos, hace que sea posible desarrollar un sistema de telepresencia robótica práctico sin requerir inversiones prohibitivas. De esta forma, se demuestra que la telepresencia inmersiva no es exclusiva de sistemas robóticos de alto coste, sino que puede implementarse sobre plataformas económicas y accesibles.

Con este trabajo se pretende validar la viabilidad de utilizar realidad virtual y robótica de bajo coste como base para un sistema de telepresencia, que facilite colaboraciones remotas más efectivas y con mayor sensación de inmersión que las alternativas actuales basadas en videollamadas convencionales.

1.2. Objetivo general

El objetivo general de este trabajo es el desarrollo de un sistema de telepresencia robótica que permita a un operador interactuar de forma remota con un entorno real mediante el uso de un robot móvil Turtlebot 2 y una interfaz de realidad virtual basada en dispositivos HMD y sus respectivos controles, en este caso las Meta Quest 2. El sistema deberá permitir no solo la teleoperación del robot, sino también la transmisión bidireccional de información sensorial y multimedia (audio y vídeo), reproduciendo un escenario equivalente a una interacción presencial. Para ello, se plantea la integración de tecnologías de robótica (ROS), simulación (Gazebo) y desarrollo de entornos inmersivos (Unity), con el fin de proporcionar una experiencia de control intuitiva, inmersiva y en tiempo real. Adicionalmente, se busca optimizar aspectos clave como la latencia de comunicación, la precisión en el control del robot y la accesibilidad de la interfaz, garantizando un rendimiento adecuado en *hardware* de coste moderado y manteniendo la compatibilidad con futuras ampliaciones del sistema.

1.2.1. Objetivos específicos

Para alcanzar el objetivo general planteado, se definen los siguientes objetivos específicos:

- Diseñar e implementar una interfaz de usuario accesible, clara e intuitiva dentro del entorno de realidad virtual, facilitando la interacción del operador con el sistema.

- Implementar la teleoperación del robot Turtlebot 2 desde la interfaz de realidad virtual, permitiendo el control de movimiento mediante dispositivos de entrada propios de las gafas Meta Quest 2.
- Desarrollar un sistema de comunicación multimodal de telepresencia que permita la interacción bidireccional entre el operador y los interlocutores remotos, con la menor latencia posible. Este sistema debe incluir:
 - Transmisión de vídeo desde el robot hacia el entorno de realidad virtual para la visualización del entorno.
 - Transmisión de vídeo adicional desde un dispositivo móvil hacia el operador, ampliando la perspectiva del entorno y mayormente de los interlocutores.
 - Envío de vídeo desde las gafas hacia el dispositivo móvil, representando al operador mediante un avatar virtual con *lip sync*.
 - Comunicación de audio bidireccional entre el operador y los interlocutores remotos.
- Analizar la usabilidad y rendimiento del sistema mediante pruebas, mediciones y valorando la efectividad de la teleoperación.
- Garantizar que el sistema sea modular y escalable, permitiendo su adaptación a futuros desarrollos.

1.3. Estructura del documento

Este documento se organiza en seis capítulos principales, complementados por secciones preliminares y apéndices. El Capítulo 1 presenta la motivación del proyecto, los objetivos general y específicos, la metodología seguida y el plan de trabajo con su respectivo cronograma. El Capítulo 2 recoge el estado del arte, analizando trabajos previos en teleoperación y telepresencia, el uso de realidad virtual en el control de robots móviles, las formas de integración entre ROS y Unity, los sistemas de comunicación multimedia en tiempo real, y el uso de dispositivos móviles en sistemas de este tipo. El Capítulo 3 describe el diseño general del sistema, incluyendo el diagrama de arquitectura y el análisis de costes. El Capítulo 4 detalla el desarrollo e implementación del sistema y sus partes, describiendo la plataforma *hardware* y *software* utilizada, la arquitectura general, el *stack* ROS del robot, la aplicación de realidad virtual en Unity, la aplicación Android y el entorno de simulación, junto con los problemas encontrados durante sus desarrollos. El Capítulo 5 presenta los experimentos realizados y las métricas evaluadas tanto en el entorno simulado como en el robot físico. El Capítulo 6 recoge las conclusiones del trabajo, el cumplimiento de los objetivos y las posibles líneas de trabajo futuras. Finalmente, se incluye la bibliografía y el apéndice con material adicional.

1.4. Metodología y plan de trabajo

El desarrollo de este proyecto se ha llevado a cabo siguiendo una metodología iterativa e incremental, centrada en la investigación, integración y validación progresiva de cada uno de los componentes del sistema. Debido a la naturaleza multidisciplinar del proyecto, que combina realidad virtual, robótica, simulación y comunicaciones en red, el trabajo se organizó en distintas fases de desarrollo y pruebas continuas.

En la primera etapa se realizó un estudio del entorno necesario para el proyecto, analizando las herramientas de desarrollo compatibles con el robot Turtlebot 2 y con el ecosistema ROS. Durante esta fase se identificaron diversas limitaciones derivadas de la obsolescencia de algunas tecnologías empleadas por el Turtlebot 2, como Ubuntu 16.04, ROS Kinetic o Gazebo 7, todas ellas fuera de su ciclo de vida útil desde 2021. Esto obligó a realizar múltiples pruebas de compatibilidad e integración para poder trabajar con herramientas más actuales como Unity 2022 y el dispositivo de realidad virtual Meta Quest 2.

Una vez configurado el entorno de desarrollo, se procedió a la implementación de la aplicación principal en Unity, encargada de gestionar la realidad virtual y actuar como el núcleo de integración del sistema. En esta aplicación se desarrollaron las funcionalidades relacionadas con la teleoperación del robot mediante controles VR, la recepción y visualización de vídeo, la interacción con ROS mediante ROS#, y la integración de distintos elementos de la interfaz de usuario.

Posteriormente, se configuró una máquina virtual con Ubuntu 16 y ROS Kinetic para ejecutar el entorno de simulación del Turtlebot 2 en Gazebo, así como los distintos nodos ROS necesarios para la navegación, el mapeado y la comunicación entre componentes. Además, se desarrolló una aplicación móvil con Android Studio destinada a la transmisión bidireccional de audio y vídeo con Unity mediante comunicación UDP, permitiendo incorporar capacidades de telepresencia al sistema.

El proceso de desarrollo se basó principalmente en ciclos continuos de prueba y corrección. Cada nueva funcionalidad era implementada inicialmente en un entorno de simulación y posteriormente validada mediante pruebas en Gazebo. Este enfoque permitió detectar errores de integración de forma temprana y asegurar el funcionamiento progresivo del sistema completo.

Para la gestión del código y el control de versiones se utilizó GitHub junto con GitHub Desktop, facilitando la organización de las distintas versiones del proyecto y permitiendo mantener un seguimiento de los cambios realizados durante el desarrollo.

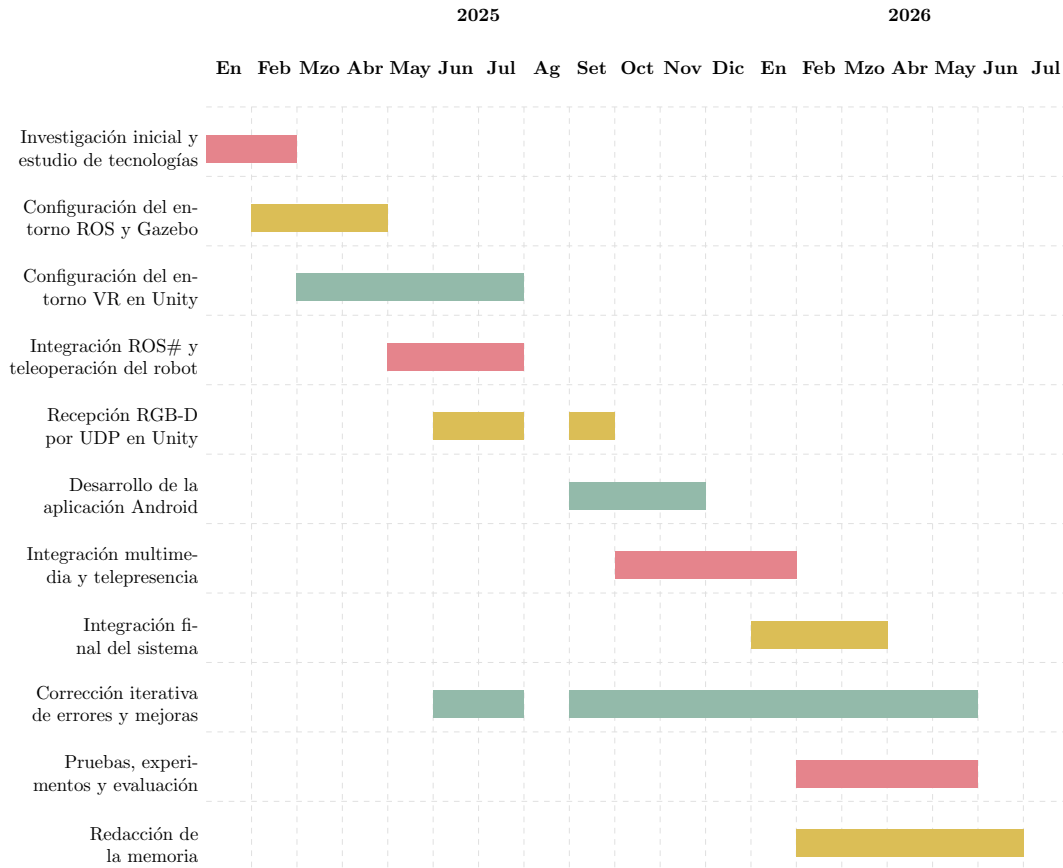
Las reuniones de seguimiento y tutorías se realizaron de manera periódica a lo largo del proyecto. En dichas reuniones se revisaba el estado actual del

desarrollo, se proponían nuevas funcionalidades y se analizaban posibles mejoras o problemas técnicos encontrados durante la implementación. La retroalimentación recibida permitió redefinir parcialmente algunos objetivos y adaptar determinadas decisiones según las necesidades.

La validación del sistema se realizó principalmente mediante pruebas sobre el entorno simulado, evaluando no solo la funcionalidad sino también aspectos como la estabilidad de la comunicación, la latencia, la tasa de imágenes por segundo y la respuesta correcta de los controles de teleoperación. Debido a la limitada disponibilidad del robot físico durante la mayor parte del desarrollo y de los limitados recursos, la mayor parte del desarrollo y las pruebas se llevaron a cabo sobre el entorno virtual, realizándose únicamente pruebas presenciales con el robot real en ocasiones muy puntuales y también al final del proyecto.

En cuanto a la planificación temporal, el desarrollo del proyecto se extendió desde enero de 2025 hasta junio de 2026. Durante los primeros meses se dedicó un mayor esfuerzo a la configuración del entorno y resolución de problemas de compatibilidad entre tecnologías. Posteriormente se abordó el desarrollo de la aplicación VR, la integración con ROS y Gazebo y las comunicaciones multimedia. Finalmente, las últimas fases del proyecto se centraron en la realización de pruebas, optimización del sistema y redacción de la memoria y presentación.

1.4.1. Cronograma



1.4.2. Explicación por tarea

- **Investigación inicial y estudio de tecnologías:** Durante esta fase se realizó un análisis inicial de las tecnologías necesarias para el desarrollo del proyecto, incluyendo el estudio del robot Turtlebot 2, ROS Kinetic, Gazebo, Unity y el dispositivo de VR Meta Quest 2. También se investigaron distintas alternativas de comunicación entre sistemas y herramientas compatibles con el entorno de desarrollo.
- **Configuración del entorno ROS y Gazebo:** En esta etapa se preparó la máquina virtual con Ubuntu 16 y ROS Kinetic, configurando los distintos paquetes necesarios para la simulación del Turtlebot 2 en Gazebo. Además, se realizaron pruebas de funcionamiento relacionadas con navegación, mapeado y comunicación mediante rosbbridge.
- **Configuración del entorno VR en Unity:** Se desarrolló la base del entorno de realidad virtual utilizando Unity y el paquete XR Interaction

Toolkit. Durante esta fase se configuró el visor Meta Quest 2, los sistemas de interacción VR y la estructura principal de la aplicación encargada de integrar el resto de componentes del sistema.

- **Integración ROS# y teleoperación del robot:** En esta fase se implementó la comunicación entre Unity y ROS mediante el paquete ROS#, permitiendo controlar el movimiento del robot desde el entorno VR. También se realizaron pruebas relacionadas con la teleoperación y la recepción de información procedente del entorno ROS para incluir en las interfaces.
- **Recepción RGB-D por UDP en Unity:** Se desarrolló el sistema encargado de recibir en Unity las imágenes RGB y de profundidad enviadas desde ROS mediante comunicación UDP. Durante esta etapa se realizaron pruebas de transmisión, visualización y sincronización de los datos recibidos.
- **Desarrollo de la aplicación Android:** Se implementó una aplicación Android destinada a la transmisión bidireccional de audio y vídeo mediante UDP. La aplicación permite completar el sistema de telepresencia manteniendo una comunicación multimedia entre los dispositivos.
- **Integración multimedia y telepresencia:** En esta fase se integraron las distintas funcionalidades multimedia dentro de la aplicación principal, incluyendo la comunicación de audio y vídeo, el sistema de telepresencia y la representación del avatar virtual utilizado durante la interacción.
- **Integración final del sistema:** Se realizó la integración conjunta de todos los componentes desarrollados previamente, incluyendo el entorno VR, ROS, Gazebo, las comunicaciones multimedia y la teleoperación del robot. Esta fase estuvo centrada principalmente en resolver problemas de compatibilidad y sincronización entre sistemas.
- **Corrección iterativa de errores y mejoras:** A lo largo de gran parte del desarrollo se realizaron tareas continuas de depuración, corrección de errores y mejora de funcionalidades. Este proceso iterativo permitió adaptar progresivamente el sistema a los distintos problemas técnicos encontrados durante el proyecto.
- **Pruebas, experimentos y evaluación:** Se llevaron a cabo pruebas funcionales tanto en el entorno simulado como, de forma limitada, sobre el robot físico. Además, se realizaron experimentos relacionados con estabilidad, latencia, tasa de imágenes por segundo y comportamiento general del sistema.
- **Redacción de la memoria:** Durante las últimas fases del proyecto se elaboró la memoria final del Trabajo Fin de Grado, documentando tanto el

proceso de desarrollo como las decisiones técnicas, pruebas realizadas y resultados obtenidos. Además, gran parte del tiempo se dedicó a la resolución de problemas de compatibilidad asociados al uso de *software* y *hardware* basados en tecnologías desfasadas sin soporte.

2

Estado del arte

En este capítulo se presenta un análisis de las tecnologías y trabajos previos a este proyecto. Se estudiarán los principales enfoques existentes en teleoperación y telepresencia robótica, la aplicación de la realidad virtual al control de robots móviles, las arquitecturas de integración entre ROS y motores gráficos como Unity, y los sistemas de comunicación multimedia en tiempo real.

2.1. Teleoperación, telepresencia y telerrobótica

Dentro de la literatura científica, la telerrobótica distingue habitualmente dos conceptos relacionados, la teleoperación, centrada en el control y ejecución de tareas mediante un robot remoto, y la telepresencia, que busca además proporcionar al usuario la sensación de encontrarse físicamente en el entorno que controla. El objetivo del conjunto de estas dos dinámicas es permitir a un operador humano realizar tareas en entornos remotos, peligrosos o con mayor comodidad, reduciendo la exposición humana directamente, pero sin prescindir de la inmersión, comunicación y control [15, 27].

El origen histórico de estos sistemas se remonta a mediados del siglo XX, cuando Raymond Goertz Goertz [7] desarrolló en 1947 los primeros telemanipuladores maestro-esclavo para la industria nuclear estadounidense. Estos sistemas puramente mecánicos sentaron las bases conceptuales de la teleoperación moderna. Con la evolución de la electrónica y las telecomunicaciones, la teleoperación fue adquiriendo mayor capacidad de control, automatización y percepción remo-

ta, extendiéndose durante las décadas siguientes a aplicaciones más avanzadas y especializadas en áreas como la medicina o lo militar.

En la actualidad, los sistemas de teleoperación se utilizan en una gran variedad de campos, como la exploración espacial, cirugía robótica como el trabajo de Marecaux et al. Marescaux et al. [13], teleasistencia, educación remota, como el trabajo de Botev et al. [3], etc.

Uno de los problemas principales de la teleoperación clásica basada en monitores es la percepción limitada debido a la bidimensionalidad del entorno remoto. El operador debe realizar una interpretación mental de la escena que observa y posteriormente transformarla en acciones de control, lo que aumenta la carga cognitiva y reduce la conciencia situacional [10]. Este concepto hace referencia a la capacidad del operador para comprender correctamente el estado del entorno y anticipar eventos futuros. Cuando el sistema de visualización es deficiente, esta capacidad disminuye significativamente, provocando mayores tasas de error y fatiga cognitiva.

A las limitaciones de percepción se suman las restricciones técnicas propias de la comunicación en remoto. La latencia de red constituye uno de los problemas más críticos: un retraso significativo entre la acción del operador y la respuesta visual del robot hace el control menos intuitivo y aumenta el riesgo de colisión o de toma de decisiones errónea. Otros factores relevantes son la calidad de la retransmisión de vídeo y audio, la pérdida de paquetes, las limitaciones de ancho de banda y los problemas de sincronización entre distintos tipos de datos [21].

En el ámbito académico, la plataforma Turtlebot se ha convertido en el estándar para la enseñanza e investigación con ROS en las universidades. Turtlebot 2, en particular, continúa siendo ampliamente utilizado pese a basarse en tecnologías de la generación anterior como lo son Ubuntu 16.04, ROS Kinetic y sensores RGB-D como Orbbec Astra, dado que su disponibilidad en laboratorios universitarios es elevada y su costo de adquisición es reducido [18]. La documentación oficial del robot describe sus capacidades de navegación y su integración nativa con el ecosistema ROS, incluyendo paquetes de SLAM como gmapping y suficiencia de navegación autónoma.

2.2. Realidad virtual en robótica y teleoperación

La realidad virtual se ha consolidado en los últimos años como una de las tecnologías más relevantes para la interacción entre humano y robot. Los visores de realidad virtual permiten al operador visualizar el entorno remoto de una manera más inmersiva, utilizando en muchos casos una visión estereoscópica,

directa con lo que ve el propio robot, adicionalmente desacoplando la dirección de la visión del movimiento físico del robot. Estudios comparativos han demostrado de manera consistente que las interfaces de VR reducen la carga cognitiva del operador respecto a pantallas planas durante tareas complejas de teleoperación [10, 25].

Widagdo et al. [27] realizaron un estudio sobre las preferencias y el confort de usuarios inmersos en un robot de telepresencia, identificando que la sensación de presencia está estrechamente relacionada con la calidad visual del video de la cámara, la latencia percibida y la correspondencia entre los movimientos de cabeza del operador y la orientación de la cámara del robot. Sus resultados muestran que los usuarios prefieren sistemas con latencia por debajo de los 100 ms para mantener una experiencia cómoda e inmersiva.

Mimnaugh et al. [15] analizaron las preferencias de movimiento de usuarios en sistemas de telepresencia inmersiva, concluyendo que los operadores tienden a preferir movimientos suaves y predecibles del robot sobre movimientos bruscos, independientemente de si el control es directo o con autonomía parcial. Este hallazgo resulta relevante para la elección de los parámetros de velocidad lineal y angular del sistema desarrollado en este trabajo.

Entre los sistemas más avanzados en la literatura destaca NimbRo Avatar [20], una plataforma de telepresencia inmersiva con telemanipulación y retroalimentación de fuerza háptica. Este sistema integra un visor VR, seguimiento de movimiento completo del cuerpo del operador y actuadores capaces de replicar esos movimientos en un robot humanoide remoto, como se puede ver en la figura 2.3. Aunque su complejidad y coste están muy lejos del alcance de un TFG académico, ilustra el alcance y las capacidades de estos conceptos cuando se disponen de los recursos adecuados.



Figura 2.1: *
Imagen robot NimbRo

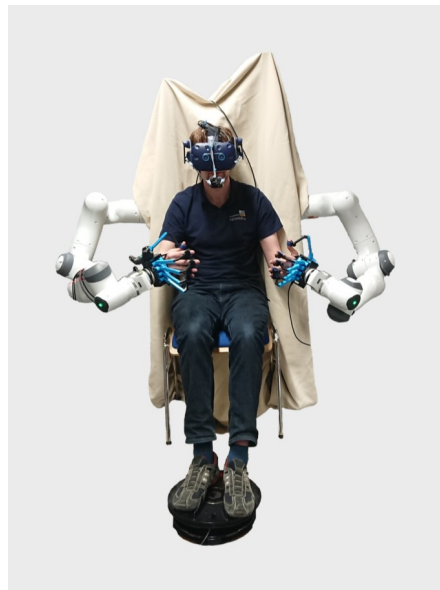


Figura 2.2: *
Imagen operador NimbRo

Figura 2.3: Sistema NimbRo Avatar (figura extraída de [20]).

En el otro extremo, en el ámbito más accesible, BUT Telepresence [21] propone un marco de trabajo de código abierto para transmisión en línea estereoscópica de vídeo con baja latencia e instrumentación de latencia extremo a extremo. Este trabajo es especialmente relevante porque demuestra que es posible construir sistemas de telepresencia funcionales con *hardware* de consumo y *software* libre. Los autores reportan latencias de extremo a extremo inferiores a 60 ms en condiciones de red local. Se puede ver el robot y la interfaz de de métricas en la figura 2.6.



Figura 2.4: *
Imagen robot BUT
Telepresence

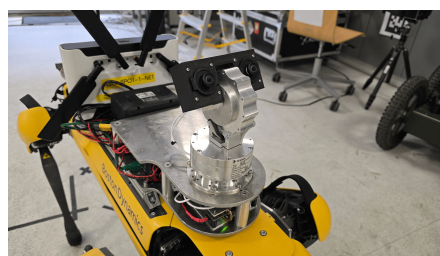


Figura 2.5: *
Imagen interfaz BUT
Telepresence

Figura 2.6: Sistema BUT Telepresence (figura extraída de [21]).

La comparativa entre realidad virtual y realidad mixta para teleoperación ha sido analizada por Cruz Ulloa et al. [5], evaluando ambas opciones en tareas de con robots con patas y un brazo robótico. Sus resultados sugieren que la realidad virtual proporciona mayor sensación de presencia, mientras que la realidad mixta facilita la conciencia del entorno físico inmediato del operador. Para un sistema de teleoperación y telepresencia de un robot móvil como el presentado aquí, la realidad virtual resulta más apropiada ya que el operador no necesita percibir su entorno físico durante el manejo del robot.

Diversos trabajos destacan además la importancia de ofrecer múltiples perspectivas visuales durante la teleoperación inmersiva. Aunque la vista en primera persona maximiza la sensación de presencia, perspectivas externas pueden mejorar significativamente la comprensión espacial del entorno y reducir la carga cognitiva en determinadas tareas de navegación. El sistema desarrollado en este proyecto adopta igualmente esta filosofía, integrando diferentes modos de visualización.

El trabajo de Dugan et al. [6] describe un sistema de telepresencia utilizando una cabeza robótica humanoide con visión estereoscópica y audio binaural. Los autores demuestran que la correspondencia entre la orientación de cabeza del operador y la de la cámara del robot mejora significativamente la capacidad de comprensión del operador de las distancias y del entorno.

2.3. Integración de ROS-Unity e infraestructuras legacy y modernas

ROS (Robot Operating System) [17] es el *middleware* estándar en investigación robótica. Aunque su nombre indique lo contrario, ROS no es un sistema operativo convencional, sino un conjunto de herramientas y librerías utilizadas para el desarrollo de aplicaciones robóticas distribuidas. Su arquitectura modular basada en nodos que se comunican mediante mecanismos de subcriptores y publicadores facilita la reutilización de *software* y la interoperabilidad entre componentes. ROS permite intercambiar información usando tópicos, servicios y acciones, ofreciendo gran flexibilidad para crear sistemas robóticos complejos.

La integración de ROS con motores gráficos como Unity presenta desafíos técnicos, ya que ambos sistemas tienen arquitecturas distintas. ROS opera de forma asíncrona mediante intercambio de mensajes en un modelo distribuido, mientras que Unity utiliza un bucle de renderizado síncrono orientado a aplicaciones gráficas en tiempo real. Para resolver este problema surgieron diferentes soluciones.

Rosbridge [23] es el protocolo más utilizado para este propósito. Expone la in-

formación de ROS a través de *WebSockets* usando mensajes codificados en JSON, permitiendo que aplicaciones externas publiquen y se suscriban a tópicos ROS sin necesidad de ejecutar código nativo. Sobre esta base se desarrolló ROS# [1], la librería en C# que permite a Unity interactuar con ROS. ROS# proporciona clases autogeneradas compatibles con los tipos de mensajes estándar de ROS, aunque también ofrece soporte para crear clases y mensajes personalizados.

El trabajo de Codd-Downey et al. [4] fue uno de los primeros en abordar la integración entre ROS y Unity para teleoperación inmersiva. Los autores propusieron un *middleware* basado en *sockets* que permitía intercambiar datos de posición, sensores y comandos de velocidad entre ambos entornos, antes de que existieran herramientas consolidadas como ROS#. Este trabajo estableció los cimientos sobre los que se han construido posteriormente numerosos sistemas de integración ROS-Unity.

Whitney et al. [26] desarrollaron ROS Reality, un *framework* completo para teleoperación y manipulación robótica mediante realidad virtual con *hardware* de consumo, como se puede ver en la figura 2.8. El sistema integra visualización tridimensional del robot, datos de sensores en tiempo real y control mediante mandos VR, todo ello sobre ROS y Unity. Un trabajo complementario de los mismos autores [25] comparó la teleoperación utilizando una interfaz de escritorio tradicional frente a controles de realidad virtual, concluyendo que la interfaz VR mejora significativamente el tiempo de completado de tareas y reduce además los errores.

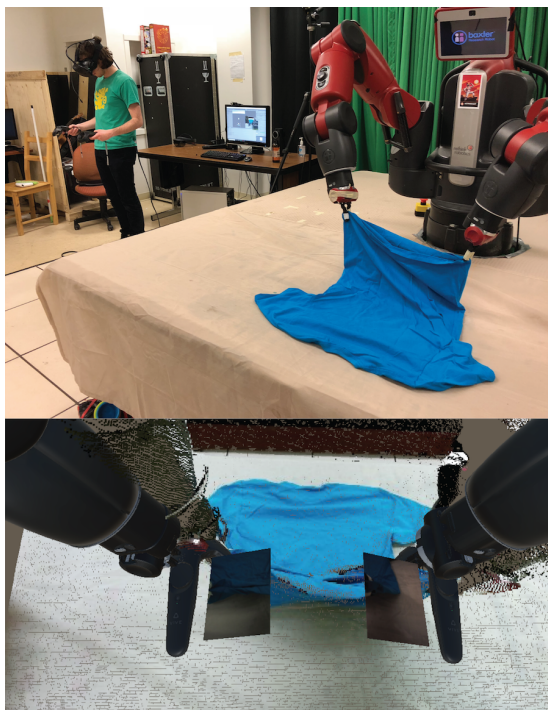


Figura 2.7: *
Imagen ROS Reality

Figura 2.8: Sistema ROS Reality (figura extraída de [26]).

Hernández et al. [8] presentaron un sistema de teleoperación robótica basado en Unity y ROS utilizando *rosbridge* como vía de conexión, publicando comandos desde Unity y visualizando la cámara del robot en el entorno virtual. Sus resultados demostraron que, para aplicaciones de teleoperación que no requieren manipulación de alta precisión, *rosbridge* y ROS# constituyen soluciones robustas y suficientemente eficientes.

Una alternativa a ROS# para sistemas más modernos es Unity Robotics Hub [22], el paquete oficial desarrollado por Unity Technologies en colaboración con Open Robotics. Este paquete ofrece soporte nativo para ROS 2, importación de modelos URDF y una comunicación más eficiente basada en TCP en lugar de *WebSockets*. Sin embargo, su compatibilidad con ROS 1 es limitada. Esta situación refleja uno de los problemas habituales en robótica académica, la coexistencia entre plataformas *hardware legacy* y herramientas *software* modernas. Muchos laboratorios universitarios continúan utilizando robots desarrollados sobre ROS 1 y distribuciones antiguas de Ubuntu debido a su disponibilidad y coste reducido, pese a que gran parte del desarrollo actual se concentra en ROS 2 y herramientas más recientes.

En este contexto, arquitecturas de desacoplamiento basadas en *middleware* in-

termedio permiten integrar tecnologías contemporáneas sin modificar el entorno *software* original del robot. En el presente proyecto, *rosbridge* cumple precisamente esta función, permitiendo que el Turtlebot 2 ejecute ROS Kinetic sobre Ubuntu 16.04 mientras Unity funciona de forma independiente en Windows 11. Esta separación facilita la integración de herramientas modernas de realidad virtual manteniendo intacta la infraestructura *software* original del robot.

La utilización de simuladores como Gazebo [19] junto a ROS constituye además una práctica estándar en el desarrollo robótico académico. Gazebo permite simular robots, sensores y entornos físicos con comportamiento relativamente realista, facilitando la validación de algoritmos antes de desplegarlos sobre *hardware* real. Del mismo modo, el uso de máquinas virtuales o entornos virtualizados facilita la reproducción de configuraciones *software* antiguas durante el desarrollo y las pruebas experimentales. En el contexto de este trabajo, Gazebo fue utilizado durante las fases iniciales de validación del sistema aprovechando los modelos de simulación incluidos en los paquetes oficiales de Turtlebot 2.

Finalmente, el TFG de Holdorf [11] abordó la generación automática de ficheros de interfaz ROS-Unity, un problema de infraestructura que también resulta relevante cuando se trabaja con mensajes personalizados en proyectos académicos de integración entre ROS y motores gráficos.

En conjunto, la literatura muestra que *rosbridge* y ROS# continúan siendo soluciones ampliamente utilizadas para integrar ROS 1 con entornos VR basados en Unity, especialmente en contextos académicos donde persisten plataformas robóticas *legacy*.

2.4. Plataformas de telepresencia inmersiva con robot móvil

Con el avance de la robótica cada vez es más común el desarrollo de trabajos que combinan la los robots móviles con interfaces de realidad virtual para telepresencia. A continuación, revisamos algunos de estos relevantes proyectos.

Tovar Martínez [24] desarrolló en la Universidad de Alicante una interfaz inmersiva de realidad virtual para teleoperación de robots móviles. Su sistema integra Unity, ROS y un visor VR para controlar un robot diferencial, visualizando la cámara del robot en el entorno virtual. El trabajo identifica como principales limitaciones la latencia de la transmisión de vídeo mediante ROS y los problemas de sincronización entre la imagen y el movimiento del robot.

Penalva Macià [16], también de la Universidad de Alicante, desarrolló un sistema de teleoperación mediante reconocimiento de gestos con las manos, eliminando la necesidad de mandos físicos, utilizando también ROS y un robot Turtlebot, en

concreto la versión 3, la siguiente a la usada en este TFG.

Reality Fusion [12] es uno de los trabajos más completos en este ámbito, se puede ver el resultado en la figura 2.10. Los autores proponen una arquitectura de telepresencia inmersiva en tiempo real que fusiona múltiples fuentes de datos: vídeo RGB, profundidad y odometría, todo ello representado en un entorno VR. El sistema incorpora técnicas de fusión volumétrica para reconstruir el entorno tridimensional en tiempo real, alcanzando representaciones densas actualizadas a varios fotogramas por segundo. Este trabajo es especialmente relevante como referencia de rendimiento, aunque su complejidad computacional supera considerablemente los recursos disponibles en un contexto académico con el Turtlebot 2.

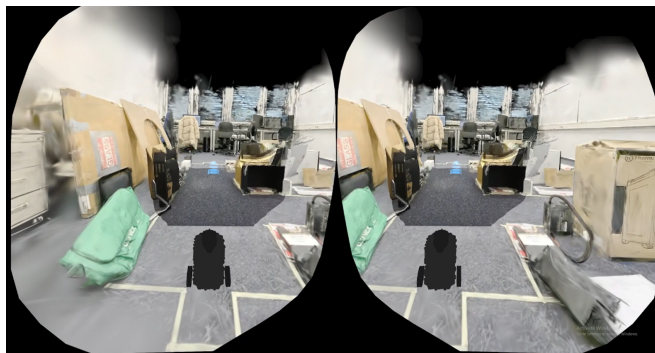


Figura 2.9: *
Imagen Reality Fusion

Figura 2.10: Sistema Reality Fusion (figura extraída de [12]).

Hernández Gobertti [9] desarrolló un sistema de teleoperación inmersiva de brazo robótico con reconocimiento de agarre y retroalimentación háptica. Aunque el tipo de robot es diferente, la arquitectura de comunicación entre Unity y ROS y el uso de visores VR son comparables con los del presente trabajo, demostrando la capacidad conjunta de este grupo de herramientas.

2.5. Comunicaciones multimedia en tiempo real

La transmisión de audio y vídeo en tiempo real es un componente esencial en cualquier sistema de telepresencia. La calidad de esta transmisión influye directamente sobre la experiencia del operador, ya que cualquier deficiencia en la tasa de fotogramas, latencia o desincronización puede entorpecer gravemente la capacidad de control y la sensación de inmersión.

Desde el punto de vista del protocolo de transporte, existe un compromiso fundamental entre fiabilidad y latencia. TCP garantiza la entrega ordenada y sin pérdidas de todos los paquetes, pero los mecanismos de control que introduce aumentan la latencia variable. UDP, en cambio, es un protocolo que no garantiza la entrega ni el orden de los paquetes, pero minimiza la latencia al eliminar estos mecanismos. Para aplicaciones multimedia interactivas en tiempo real UDP es la opción estándar [21].

La transmisión de flujos multimedia en estos sistemas suele basarse en UDP tanto para vídeo como para audio, ya que permite priorizar la latencia frente a la fiabilidad absoluta, asumiendo que la pérdida ocasional de paquetes es preferible a la introducción de retardos perceptibles.

En cuanto a la codificación de vídeo, la compresión JPEG es una solución ampliamente utilizada en sistemas de telepresencia debido a su simplicidad y bajo coste computacional. Aunque códecs más avanzados como H.264 ofrecen una mayor eficiencia de compresión, requieren una mayor complejidad de implementación. JPEG representa un compromiso adecuado en escenarios con resoluciones moderadas y requisitos de baja latencia [26].

De forma similar, el audio en estos sistemas se transmite frecuentemente en formato PCM sin compresión cuando se prioriza la simplicidad y la baja latencia. Con frecuencias de muestreo típicas de 16 kHz y 16 bit mono, este enfoque resulta viable en entornos con suficiente ancho de banda, evitando la sobrecarga de codificación adicional.

Otro aspecto crítico en sistemas multimedia es la sincronización entre diferentes flujos de datos. Mientras que los datos ligeros como la odometría o los comandos de movimiento se transmiten prácticamente sin retraso, los flujos más pesados como el audio o el vídeo requieren mucho más ancho de banda. Esto puede provocar desincronización entre las diferentes transmisiones del sistema [12].

El trabajo de Botev et al. [3] sobre telepresencia inmersiva para educación remota muestra que la bidireccionalidad del audio es valorada muy positivamente por los usuarios y contribuye significativamente a la sensación de inmersión.

2.6. Integración de dispositivos móviles en sistemas de telepresencia

El uso de teléfonos inteligentes como dispositivos auxiliares en sistemas de teleoperación y telepresencia se ha incrementado notablemente en los últimos años. Su atractivo principal es que integran de serie características prácticas y técnicas como cámara, micrófono, altavoz, unidad de procesamiento y conectividad

inalámbrica, todo ello a un coste muy accesible. Berengueres et al. Berengueres [2] analizan esta unión entre estos dispositivos inteligentes y robots, argumentando que los primeros pueden reducir drásticamente el coste de las plataformas robóticas al reutilizar tanto el *hardware* como el ecosistema de desarrollo de aplicaciones ya existentes.

Desde una perspectiva más amplia, Youssef et al. Youssef et al. [28] hacen una revisión del estado actual de la telepresencia robótica, identificando la integración de dispositivos móviles de consumo como una de las tendencias actuales en los sistemas de bajo coste.

En este contexto, los dispositivos móviles pueden actuar como nodos auxiliares dentro del sistema de telepresencia, proporcionando capacidades de captura de audio y vídeo, así como procesamiento local y conectividad inalámbrica. Este tipo de arquitecturas aprovecha la disponibilidad de *hardware* ya existente en dispositivos de consumo, reduciendo la necesidad de sensores adicionales en el sistema robótico.

Una aproximación ampliamente utilizada para la transmisión de audio y vídeo en estos sistemas consiste en el uso de WebRTC. Tal y como describen Melendez et al. Melendez-Fernandez et al. [14], WebRTC proporciona mecanismos integrados de comunicación en tiempo real, incluyendo sincronización audiovisual, control de congestión y transmisión adaptativa, lo que simplifica el desarrollo de aplicaciones de telepresencia.

Sin embargo, su integración puede presentar limitaciones dependiendo del entorno de desarrollo. En particular, algunos motores de desarrollo utilizados en sistemas de telepresencia no disponen de soporte nativo para la recepción y renderizado de flujos WebRTC, lo que puede dificultar su adopción en determinados escenarios.

2.7. Comparativa y posicionamiento del proyecto

Los trabajos más relevantes aquí vistos comparten varias características del sistema desarrollado en este TFG. Muchos que integran VR y ROS para teleoperación de robots móviles, pero la mayoría de ellos presentan alguna de las siguientes limitaciones respecto al sistema aquí desarrollado:

1. La ausencia de audio bidireccional, omiten la comunicación de audio, limitando la experiencia a únicamente la parte visual. El audio bidireccional es un componente que mejora significativamente la sensación de presencia en el lugar del robot y, además, la practicidad del sistema aumenta [3, 6].

2. Trabajos como NimbRo Avatar [20] o Reality Fusion [12] requieren *hardware* especializado o de un alto coste, siendo difícilmente reproducibles con *hardware* de consumo en un ámbito académico.
3. Una parte significativa de los trabajos recientes utilizan ROS 2 y sistemas modernos, lo que los hace incompatibles con el Turtlebot 2 de forma nativa.
4. Ninguno de los trabajos revisados integra un teléfono Android como nodo adicional del sistema de telepresencia, proporcionando cámara y audio independientes del robot.

La principal aportación del presente proyecto no reside en la creación de nuevos algoritmos o la implementación de inteligencia artificial, sino en integrar de forma funcional y estable múltiples tecnologías gratuitas para lograr implementar en una plataforma robótica de bajo coste, compatible con *software legacy*, un sistema de telepresencia y teleoperación reproducible y ampliable.

3

Diseño

En este capítulo se describe el diseño general del proyecto, la interacción entre sistemas y el presupuesto necesario para realizarlo.

3.1. Diseño del sistema

El diseño del sistema se concibe como una arquitectura distribuida orientada a soportar teleoperación bidireccional en tiempo real entre un operario remoto y una plataforma robótica móvil. El principal reto arquitectónico reside en coordinar dispositivos heterogéneos: robot, ordenador central, visor de realidad virtual y dispositivo móvil Android, manteniendo baja latencia, modularidad y compatibilidad entre entornos *software* muy distintos.

Para ilustrar esta estructura global, la Figura 3.1 muestra el diagrama de arquitectura del sistema. Hay que tener en cuenta que este diagrama cambia cuando se utiliza el entorno simulado, ya que el robot existe dentro de una máquina virtual de Ubuntu 16 junto con un espacio virtual, gracias a Gazebo.

El sistema se estructuró bajo un enfoque modular, donde cada componente mantiene responsabilidades claramente delimitadas y se comunica mediante interfaces de red bien definidas. Esta separación reduce el acoplamiento entre subsistemas y permite modificar elementos *hardware* o *software* sin rediseñar la arquitectura global.

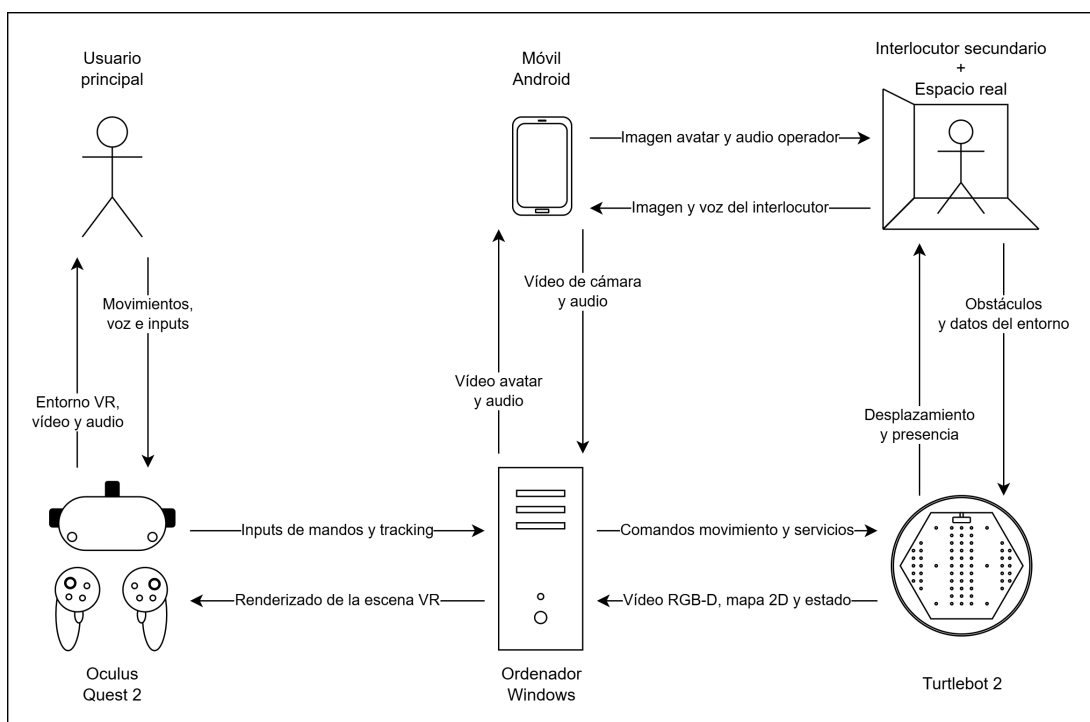


Figura 3.1: Diagrama de la arquitectura del sistema.

3.2. Criterios de diseño

La arquitectura final viene condicionada por restricciones técnicas concretas del proyecto:

- **Compatibilidad con *software legacy*:** El Turtlebot2 opera sobre ROS Kinetic y Ubuntu 16.04, un entorno sin soporte oficial y con fuertes restricciones de actualización. La solución debía integrarse con esta plataforma sin alterar su *stack* base.
- **Heterogeneidad de plataformas:** El sistema combina Linux, Windows y Android, por lo que era necesario desacoplar los mecanismos de comunicación de las implementaciones internas de cada entorno.
- **Baja latencia en tiempo real:** La naturaleza interactiva de la teleoperación obliga a priorizar tiempos de respuesta reducidos tanto en vídeo como en control.

3.3. Decisiones arquitectónicas

A partir de estos criterios se adoptaron tres decisiones principales.

3.3.1. Topología centralizada

Se optó por una topología en estrella donde el ordenador principal actúa como nodo de integración entre todos los subsistemas. Esta aproximación simplifica el encaminamiento de mensajes y evita dependencias directas entre dispositivos que operan sobre plataformas incompatibles. Frente a una arquitectura totalmente distribuida, esta decisión reduce complejidad de sincronización y facilita la depuración del sistema.

3.3.2. Separación por tipo de tráfico

El sistema distingue conceptualmente entre dos tipos de comunicación: tráfico de control y estado y tráfico multimedia de alta frecuencia. La separación responde a que ambos presentan requisitos distintos de fiabilidad, ancho de banda y tolerancia a pérdidas. Mantener canales independientes evita compromisos de diseño y mejora la eficiencia general del sistema.

3.3.3. Desacoplamiento del subsistema móvil

El dispositivo Android se gestiona como un nodo independiente del robot. En lugar de considerarlo un periférico embebido, se integra como un subsistema autónomo. Esta decisión facilita ampliar capacidades de comunicación sin modificar la arquitectura base del robot.

3.4. Flujo de interacción

El intercambio de información puede entenderse mediante dos ciclos principales:

1. Ciclo de percepción: Los sensores del robot y el dispositivo móvil generan flujos continuos de datos hacia el ordenador central.
2. Ciclo de acción: Las órdenes del operario son procesadas por el ordenador central y transmitidas a los actuadores correspondientes.

Esta organización permite mantener independencia funcional entre adquisición, procesamiento y actuación, simplificando la escalabilidad y la mantenibilidad del sistema.

3.5. Presupuesto y costes

En esta sección se detalla el análisis económico del proyecto, estructurado en los supuestos costes en materiales (*hardware* y *software*) y de personal.

Para el cálculo de la mano de obra, se toma como referencia la dedicación teórica asignada a un Trabajo de Fin de Grado de 12 créditos ECTS, lo que equivale a un total de 300 horas de trabajo estimadas. Cabe destacar que se ha dedicado un tiempo bastante superior al indicado debido a la curva de aprendizaje de las tecnologías empleadas y a los problemas derivados de su obsolescencia. Se ha considerado como una inversión en formación y, por lo tanto, se ha omitido del cómputo final para no inflar el presupuesto. El proyecto se ha desarrollado íntegramente utilizando *hardware* preexistente (de propiedad personal o cedido por el laboratorio de la universidad) y *software* de acceso libre o licencias académicas. Por lo tanto, el coste directo de adquisición para la realización de este trabajo ha sido nulo.

No obstante, con el fin de reflejar la viabilidad económica y el coste que supondría replicar este sistema en un entorno real, se detalla a continuación el valor comercial estimado de los recursos empleados y el uso aproximado que se les ha dado durante el desarrollo. Véase que de disponer del PC de sobremesa en el mismo espacio que el Turtlebot 2, no sería necesario contar con un portátil para las pruebas presenciales.

Los costes de personal se calculan asumiendo el rol de un ingeniero junior con una tarifa de mercado competitiva y realista de 13,50 €/h (coste bruto por hora). Como se mencionó anteriormente, se acotan las horas a los estándares de la titulación, descartando las demoras asociadas a la inexperiencia con las plataformas utilizadas y los problemas derivados con el *hardware* y *software*. En conclusión, el coste total para la replicación de este proyecto asciende exclusivamente al valor del trabajo invertido, ya que todos los recursos materiales como se ha indicado anteriormente han sido cedidos, ya se tenían o eran gratuitos. Al aprovechar esto, se ha obtenido un proyecto funcional sin inversión financiera directa, la cual en caso de haber tenido que adquirir todo el material y el coste de personal, el desembolso ascendería a aproximadamente 8307,55€.

Recurso material	Uso	Valor Comercial	Coste Real
<i>Hardware de Laboratorio y Pruebas</i>			
Turtlebot 2	Plataforma robótica para despliegue	1.400,00 €	0,00 €
Meta Quest 2	Visor VR para la interfaz de usuario	349,00 €	0,00 €
Portátil para pruebas presenciales	Validación del sistema en presencial	1.000,00 €	0,00 €
<i>Hardware Personal</i>			
PC de sobremesa para desarrollo	Programación, simulación y entorno Unity	1.258,56 €	0,00 €
Smartphone Android	Pruebas secundarias y conectividad	249,99 €	0,00 €
<i>Software y Licencias</i>			
Unity 2022	Motor gráfico (Licencia Personal/Student)	0,00 €	0,00 €
ROS Kinetic + Ubuntu 20.04	Entorno del robot (Código abierto)	0,00 €	0,00 €
Librerías y paquetes externos	Integración (ROS-Sharp, OpenCV, etc.)	0,00 €	0,00 €
Total Recursos Materiales		4.257,55 €	0,00 €

Tabla 3.1: Desglose de recursos materiales, uso, valor comercial y coste real de adquisición.

Fase del Proyecto	Horas Asignadas	Precio / Hora	Coste Total
Diseño e implementación del sistema	200 h	13,50 €	2.700,00 €
Pruebas, despliegue presencial y depuración	70 h	13,50 €	945,00 €
Redacción de documentación y memoria	30 h	13,50 €	405,00 €
Total Mano de Obra	300 h	-	4.050,00 €

Tabla 3.2: Desglose de los costes de personal.

4

Desarrollo

En este capítulo se describe el proceso de implementación del sistema de telepresencia robótica, detallando las decisiones de diseño tomadas, las tecnologías empleadas y el funcionamiento de cada componente desarrollado.

4.1. Plataforma de desarrollo

A continuación se detallan los elementos de *hardware* y *software* que componen este proyecto.

4.1.1. Hardware

- El **Turtlebot 2** es un robot móvil de bajo coste con un *software* de código abierto. Está equipado con una base móvil Kobuki y con una cámara RGB-D Orbbec Astra para la captura de imágenes y datos de profundidad. Este robot fue el elegido como plataforma física para la teleoperación, para ser controlado de forma remota a través de la interfaz de realidad virtual desarrollada. En la Figura 4.1 se muestra la configuración final del *hardware* con el dispositivo móvil anclado al robot.



Figura 4.1: Montaje final del Turtlebot 2 con el dispositivo móvil Android que actúa como nodo de telepresencia.

- Las gafas de realidad virtual **Meta Quest 2** (anteriormente Oculus Quest 2) fueron utilizadas para la interacción inmersiva con el entorno de teleoperación. Estas gafas ofrecen una visualización estereoscópica, permitiendo al usuario controlar el Turtlebot y recibir *feedback* visual desde la cámara del robot. Cuentan con dos paneles LCD de 1832 x 1920 píxeles cada uno, de 90 Hz de frecuencia de refresco. Cuenta con un procesador Qualcomm Snapdragon XR2, 6 GB de memoria ram, un sistema de seguimiento *inside-out* de 6 grados de libertad (6DoF) y 503 gramos de peso. Cuentan con altavoces estéreo para sonido espacial y micrófono. Tiene WiFi 6 y Bluetooth 5.0, y se puede conectar al PC utilizando Meta Horizon Link (anteriormente Meta Quest Link) mediante un cable USB-C 3.0 o de forma inalámbrica con Air Link.
- Para el desarrollo se utilizó un **ordenador personal de sobremesa** con el sistema operativo Windows 11 Pro, con una tarjeta gráfica dedicada Nvidia RTX 5060 Ti 16 GB VRAM, un procesador AMD Ryzen 7 7700, 32 GB de RAM DDR5, 1TB de almacenamiento. Este ordenador ha permitido un desarrollo fluido ya que cuenta con capacidades de sobra para poder ejecutar el proyecto completo de realidad virtual Unity a la vez que corría la máquina virtual de Ubuntu 16 en la que se simulaba el Turtlebot, y además permitía ejecutar otras tareas como Android Studio, el navegador web, procesador de textos, etc. Para las pruebas con el robot real, debido a la imposibilidad

impuesta de trabajar con él desde mi casa, se utilizaron distintos portátiles y ordenadores, lo suficientemente potentes como para ejecutar Unity, pero sin la necesidad de tener la máquina virtual.

- Se utilizó un dispositivo **móvil Android** personal para probar y ejecutar la aplicación de creada con Android Studio, usada para la comunicación audiovisual entre operador e interlocutor. El dispositivo concreto era el Xiaomi Poco X7 Pro, con Android 16. Las características necesarias para ejecutar la aplicación son que disponga de cámara frontal, micrófono, altavoz y Android 8.0 (Oreo) en adelante. Este mismo dispositivo se utilizó durante las pruebas con el robot real como punto de acceso WiFi para que los dispositivos estuvieran conectados a una misma red. También se probó la app en un dispositivo más antiguo, un Xiaomi Redmi Note 7, con la versión de Android 10, funcionando correctamente.

4.1.2. Software

- El sistema operativo Ubuntu 16.04 (Xenial Xerus) fue el utilizado tanto por el Turtlebot 2 como para la máquina virtual. Esta distribución concreta de Linux fue lanzada en 2016, y es la más adecuada para trabajar con la plataforma móvil, debido a su compatibilidad con ROS Kinetic, Gazebo 7 y los paquetes del Turtlebot.
- **Hyper-V y Oracle VirtualBox** se emplearon como plataformas de virtualización para ejecutar Ubuntu 16.04 en una máquina virtual dentro del sistema anfitrión Windows 11 Pro. Durante el desarrollo se utilizaron ambas soluciones debido a diferencias de compatibilidad con la entrada y salida de audio, la cual ya viene incluida en las *Guest Additions* de VirtualBox pero no en Hyper-V.
- **ROS** (Robot Operating System) es un *framework* y *middleware* de código abierto que proporciona la infraestructura de comunicación distribuida necesaria para desarrollar la comunicación entre los distintos componentes del Turtlebot. En este proyecto, se empleó ROS Kinetic Kame, lanzado en 2016, la distribución más adecuada y compatible con el Turtlebot 2 y para la simulación en Gazebo 7.
- **Unity** es el motor de desarrollo utilizado para crear la interfaz de realidad virtual, en concreto se usó la versión 2022.3.17f1 y posteriormente la 2022.3.62f3 por actualizaciones importantes de seguridad. Unity permitió construir el entorno visual en el que se simuló y controló el Turtlebot. Para la integración con las Meta Quest 2, se usaron el paquete XR Interaction Toolkit.

- **Android Studio** en su versión Panda 2 se ha utilizado para crear la aplicación móvil utilizada para la retransmisión audiovisual bidireccional entre operador e interlocutor. Se ha utilizado Kotlin como lenguaje de programación, siendo expresivo y más conciso que Java, la otra alternativa.
- **ROS#** es un paquete de comunicación que conecta Unity con ROS mediante el uso de *rosbridge*. Este paquete permitió que los datos de sensores, comandos de movimiento y otra información, se transmitan entre Unity y el Turtlebot.
- **Gazebo 7** es un simulador robótico que se utilizó para realizar la mayoría de las pruebas del sistema, completamente en un entorno virtual. En esta simulación, se replicarán el Turtlebot y su entorno de trabajo para verificar que la interfaz de teleoperación funcione correctamente antes de realizar pruebas en el *hardware* real. Gazebo permitirá evaluar los movimientos del robot, la transmisión de datos y la respuesta del sistema sin necesidad de utilizar el Turtlebot físico.
- **Python 2.7** se empleó como lenguaje para el desarrollo de los nodos de control y comunicación con el robot. Mediante las bibliotecas *rospy*, *OpenCV* y *NumPy* se implementaron funciones de procesamiento de imágenes, transmisión de datos por red y gestión de servicios y tópicos de ROS.

4.1.3. Herramientas adicionales

- Se utilizó **Visual Studio 2022** con el componente de desarrollo para juegos en Unity. Este fue el entorno de desarrollo integrado (IDE) para escribir, depurar y probar el código.
- **GitHub** se utilizó como sistema de control de versiones. El repositorio permitió mantener un historial de los cambios realizados durante el desarrollo y retornar a versiones anteriores en casos en los que algún cambio salió mal.

4.2. Arquitectura general del sistema

El sistema se compone de tres bloques principales que trabajan de forma coordinada:

- El robot Turtlebot2, que ejecuta el entorno ROS y los nodos necesarios para navegación, localización, gestión del robot y transmisión de vídeo.
- Una aplicación de realidad virtual desarrollada en Unity, utilizada por el operador y ejecutada en un PC con Windows.

- Una aplicación Android instalada en un dispositivo móvil, que permite la comunicación con el interlocutor remoto, recibiendo el video del avatar y el audio del usuario y enviando su propia cámara y micrófono.

La comunicación entre los distintos componentes se realiza mediante dos mecanismos. ROS Bridge se utiliza para el intercambio de información relacionada con el robot, mientras que el protocolo UDP se utiliza para la transmisión de audio y vídeo en tiempo real. Ambos mecanismos conectan los elementos a través de una red WiFi local.

4.2.1. Decisiones de diseño transversales

Durante el desarrollo del sistema fue necesario tomar una serie de decisiones de diseño que afectan a la arquitectura global. Se describen a continuación las más relevantes, junto con las alternativas consideradas y los motivos que llevaron a la elección final.

La transmisión de vídeo y audio ha realizado sobre UDP en lugar de TCP. La razón principal es el comportamiento que tienen ambos protocolos ante la pérdida de paquetes, ya que TCP garantiza la entrega, y si hay paquetes que se pierden, se reenvían, y al recibirlos se reorganizan en orden. Esto es positivo en muchos casos, pero también puede introducir latencia variable cuando hay pérdida. En cambio, UDP descarta los paquetes perdidos sin esperar, lo que mantiene una latencia baja y constante, aunque se puedan perder *frames* de forma ocasional, lo cual es aceptable para este tipo de aplicación.

Se ha utilizado JPEG frente a *codecs* de vídeo como H.264, ya que la compresión de cada *frame* se hace de forma independiente. Aunque JPEG tiene una eficiencia de compresión inferior a la de *codecs* de vídeo como H.264, que explotan la redundancia temporal entre *frames* consecutivos, JPEG tiene una ventaja, ya que *frame* es autocontenido y puede decodificarse de forma independiente. Con H.264, la pérdida de un *frame* de referencia inutiliza todos los *frames* dependientes hasta el siguiente, lo que combinado con UDP produciría errores visuales. Con JPEG, la pérdida de un paquete UDP implica simplemente no mostrar ese *frame*.

PCM crudo frente a *codecs* de audio comprimido, al igual que con el vídeo, el audio se transmite sin codificar, en formato PCM de 16 bits sin compresión adicional. A 16.000 Hz mono, el *bitrate* resultante es de 256 Kbps, asumible para la red local utilizada. Los *codecs* de audio con pérdida como Opus introducen latencia de codificación y decodificación que, aunque pequeña, se suma al resto de latencias y además incrementa la dificultad de implementación.

El uso de ROS Bridge y concretamente ROS# frente a otras alternativas se utilizó ya que ROS1 no tiene soporte nativo en Windows ni en Unity. Las alternativas eran utilizar herramientas como ROS Reality, el cuál es un proyecto

antiguo abandonado o Unity Robotics Hub, que es demasiado moderno y no tiene soporte para ROS Kinetic. ROS# expone todos los topics y servicios de ROS como una API *WebSocket* estándar, lo que permite que Unity se comuniquen con el robot sin importar el sistema operativo en el que esté.

Se ha utilizado gmapping frente a otros sistemas de SLAM para la construcción del minimapa, ya que este sistema de SLAM 2D basado en filtro de partículas viene incluido por defecto en el paquete de navegación del Turtlebot2. Aunque existen alternativas más modernas como Cartographer, gmapping es la opción más utilizada y documentada para el Turtlebot2. Además, los mapas de ocupación que genera son compatibles con el resto del sistema.

4.3. Entorno ROS en el Turtlebot2

4.3.1. Stack de software en el robot

El Turtlebot2 ejecuta Ubuntu 16.04 con ROS Kinetic. Para poner el robot en funcionamiento se lanzan secuencialmente los siguientes procesos:

1. `roslaunch Turtlebot_bringup minimal.launch` inicializa los *drivers* del *hardware* de Kobuki, motores, sensores de choque, sensores de acantilado, batería y odometría.
2. `roslaunch Turtlebot_navigation gmapping_demo.launch` lanza el *stack* de navegación con SLAM basado en gmapping, que construye un mapa de ocupación 2D a partir del láser y lo publica en el topic `/map`.
3. `roslaunch rosbridge_server rosbridge_websocket.launch` expone todos los topics y servicios de ROS como una API *WebSocket* en el puerto 9090.
4. `udp_camera_stream_robust.py` nodo personalizado que captura las imágenes de la cámara RGB-D y las transmite por UDP.
5. `Turtlebot2_robot_manager.py` nodo personalizado que centraliza la gestión del robot y expone una interfaz de control hacia Unity.

Si en vez del robot real estamos trabajando en el simulador, los dos primeros comandos cambian, ya que se lanzan los comandos de gazebo, `roslaunch Turtlebot_gazebo Turtlebot_world.launch` y `roslaunch Turtlebot_gazebo gmapping_demo.launch` respectivamente, el resto se mantiene igual.

4.3.2. Nodo de streaming de cámara RGB-D `udp_camera_stream_robust.py`

Este nodo es responsable de transmitir el vídeo de la cámara del Turtlebot2 hacia Unity en tiempo real. Se suscribe a los topics `/camera/rgb/image_raw` y `/camera/depth/image_raw`, y envía los *frames* comprimidos en JPEG por UDP hacia la IP y puertos configurados, las respectivas del ordenador en que se ejecuta Unity. La Figura 4.4 ilustra los flujos de imagen en color y profundidad que son capturados y procesados por el nodo antes de su envío.

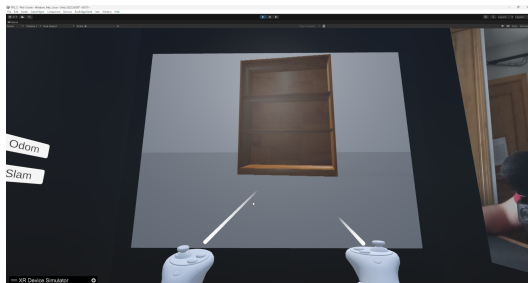


Figura 4.2: *
Imagen RGB

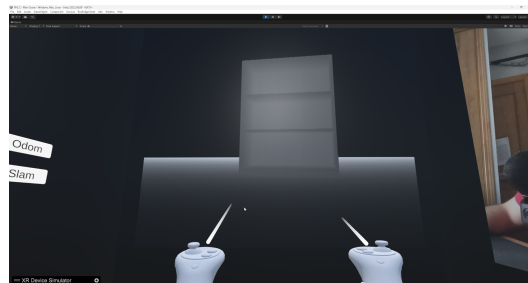


Figura 4.3: *
Imagen de Profundidad

Figura 4.4: Capturas del entorno virtual mostrando los flujos RGB y de profundidad (Depth) generados por la cámara del robot.

El modo de transmisión (RGB o profundidad) puede cambiarse dinámicamente suscribiéndose al topic `/camera_stream_mode`, que Unity publica cuando el usuario cambia de modo en la interfaz. De esta forma, el nodo solo retransmite lo que el operador está visualizando en cada momento, evitando consumo innecesario de ancho de banda.

Cada *frame* se redimensiona a 320×240 píxeles y se comprime con JPEG a calidad 30 antes de enviarse, lo que produce paquetes de tamaño inferior a 65.000 bytes, dentro del límite de un datagrama UDP estándar sin fragmentación. La frecuencia de envío se limita a 15 FPS mediante un control de tiempo, suficiente para la teleoperación y ajustada al ancho de banda disponible.

El nodo incorpora reconexión automática ante fallos del *socket*, con un tiempo de espera de 2 segundos y un retardo de reconexión de 2 segundos. Los errores de red se registran sin interrumpir la ejecución y se muestran en la consola.

4.3.3. Nodo de gestión del robot `Turtlebot2_robot_manager.py`

Este nodo centraliza funciones de monitorización y control del robot que no forman parte del *stack* estándar de ROS, proporcionando lo necesario para la integración de una interfaz simplificada en Unity. Para ello, expone los siguientes servicios:

- `/reset_slam`: reinicia el nodo gmapping y la odometría del robot, permitiendo reconstruir el mapa cuando la localización se ha degradado o el mapa generado acumula errores.
- `/reset_odometry`: reinicia únicamente la odometría del robot sin afectar al SLAM ni al mapa existente.

Además, el nodo publica periódicamente información de estado en el tópico `/robot_status` mediante un mensaje `std_msgs/String` en formato JSON. Esta información incluye el voltaje, del que se calcula el porcentaje estimado de batería según los valores máximo y mínimo, obtenidos a partir de los datos proporcionados por `/mobile_base/sensors/core`. También se obtiene la velocidad lineal y angular instantáneas calculadas a partir de la odometría recibida en `/odom`. Esta información permite supervisar de forma centralizada el estado básico de funcionamiento del robot desde la interfaz de usuario.

4.4. Entorno de realidad virtual en Unity

La aplicación de Unity es el núcleo del sistema desde el punto de vista del operador. Integra en un único entorno de realidad virtual la teleoperación del robot, la comunicación audiovisual con el interlocutor y toda la interfaz de control. Este apartado describe su implementación, organizada en tres bloques, el entorno VR y su interfaz, la comunicación con ROS y la teleoperación, y el sistema multimedia bidireccional.

4.4.1. Realidad virtual e interfaz de usuario

El paquete XR Interaction Toolkit proporciona las herramientas y componentes necesarios para implementar el entorno de realidad virtual, sin necesidad de programar desde cero las acciones básicas, ya que rastrea la cabeza (con el visor) y las manos del jugador (con los mandos), gestiona el movimiento y rotación y facilita la implementación de las interacciones con objetos y menús. El mundo virtual consta de una sala cúbica en la que se encuentra el operador. Enfrente tiene ubicada la pantalla principal que muestra la retransmisión del robot. A la derecha

se encuentra la pantalla secundaria, que muestra la imagen del dispositivo Android. Por último, a la izquierda se encuentra la interfaz de usuario. Además, para mitigar el mareo por movimiento durante la navegación, se ha aplicado *tunneling vignette*, que oscurece los bordes del campo de visión al desplazarse. La interfaz de usuario es un *canvas* en el espacio del mundo, integrada en el entorno VR. Hay dos componentes encargados de gestionar esta interfaz, uno manejando los elementos relacionados con el Turtlebot y el otro manejando la parte de Unity. La interfaz gráfica se adapta a las condiciones del entorno (simulado o real), proporcionando al operador toda la telemetría necesaria (ver Figura 4.7). Las opciones que tiene dicha interfaz son las siguientes:

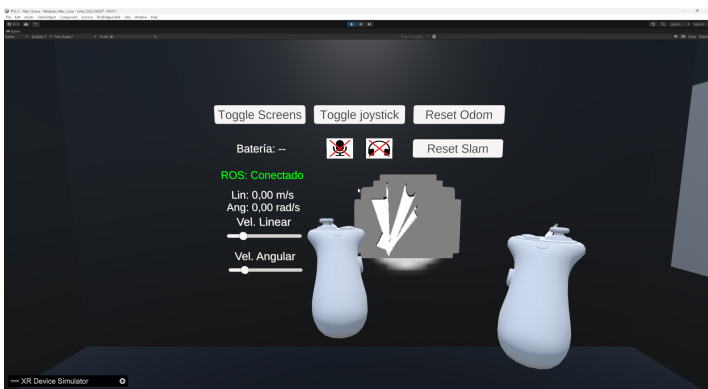


Figura 4.5: *
Interfaz en entorno simulado

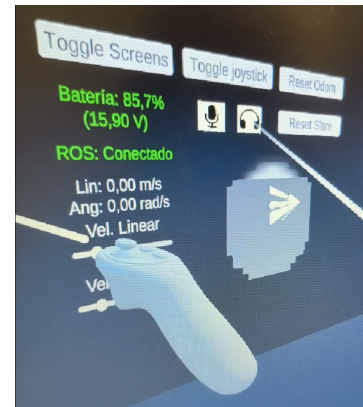


Figura 4.6: *
Interfaz en entorno real

Figura 4.7: Paneles de la interfaz de usuario en Unity, mostrando el minimapa, estado de la batería, conexión de ROS y controles rápidos.

- Gestiona el audio, con un botón para silenciar el micrófono del operador y otro botón para ensordecerse del audio del interlocutor, con iconos que reflejan el estado actual.
- Un minimapa que se contruye gracias al componente subscritor del mapa de ocupación, que procesa el mensaje `nav_msgs/OccupancyGrid` de ROS, que representa el mapa como un array unidimensional de enteros de 8 bits. Los valores posibles son 0 (celda libre), 100 (celda ocupada) y -1 (celda desconocida), que se traduce en los colores blanco, negro y gris respectivamente. El array se indexa fila a fila, de forma que la celda en la columna x y la fila y se encuentra en la posición $y \times \text{ancho} + x$. A partir de estos datos se genera la textura del minimapa.
- Permite ajustar los factores de escala de la velocidad lineal y angular del robot, y además muestra los valores actuales a los que se mueve el robot.

- Alterna el *joystick* mediante un botón, para cambiar entre mover al usuario por el entorno VR o controlar el movimiento del robot.
- Existen dos botones para acciones rápidas para reiniciar la odometría y el SLAM, por si existiera algún problema en alguno de ambos elementos o en el mapa.
- Muestra información sobre el estado del componente de Unity **RosConnector**, que es el encargado de gestionar la conexión con el Turtlebot mediante ROS#.
- Monitorea en tiempo real el nivel de batería aproximado (usando el valor del voltaje de la batería) del robot mediante un código de colores, verde por encima del 50 %, amarillo entre el 20 % y el 50 %, y rojo por debajo del 20 %.
- Además, sincroniza los hilos, garantizando que todas las modificaciones de la interfaz gráfica se ejecuten en el hilo principal de Unity. Los datos recibidos desde el suscriptor ROS, que operan en un hilo secundario, se notifican mediante un *flag volatile* y un *callback*.

4.4.2. Comunicación con ROS y teleoperación

La conexión entre Unity y ROS se establece mediante la librería ROS#, que implementa el protocolo *ROSBridge* sobre *WebSocket*. El componente **RosConnector**, incluido en el paquete, gestiona la conexión persistente con el servidor *WebSocket* del robot. Todos los demás componentes de Unity que necesitan comunicarse con ROS obtienen la referencia al *socket* a través de este componente. Cualquier componente puede ahora publicar o suscribirse a topics de ROS sin conocer los detalles de la conexión. El uso más importante de esto es el componente **VRTeleop**, que gestiona el control del robot desde el entorno de realidad virtual. Publica mensajes **Twist** en el topic `/cmd_vel` del Turtlebot2, que ROS Bridge retransmite al robot. Este componente también gestiona la entrada del operador, a través de los *Input Actions*, leyendo el valor del mando de las gafas de VR. El eje vertical se traduce en velocidad lineal y el eje horizontal en velocidad angular, con factores de escala configurables. La publicación se realiza a 5 Hz, frecuencia suficiente para la teleoperación y que evita saturar el topic. Además, se asegura que nunca se envíe el valor 0 para, evitar que se pueda parar el robot.

4.4.3. Sistema multimedia bidireccional

Para la telepresencia es esencial tener audio y video de forma bidireccional entre operador e interlocutor. Para ello se crearon varios sistemas, todos por

UDP, para enviar y recibir esta información y generar así un concepto similar a una videollamada. En la Figura 4.10 se observa la disposición de estas pantallas dentro de la sala virtual, mostrando de forma simultánea la perspectiva del robot y el *feed* de la cámara del interlocutor, tanto en pruebas simuladas como reales.

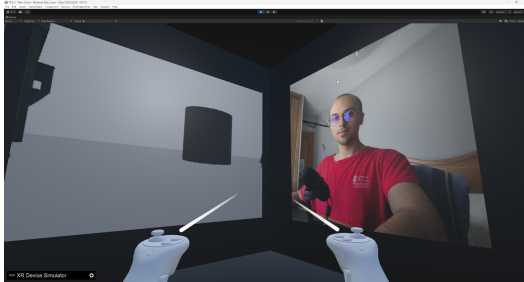


Figura 4.8: *
Telepresencia en simulación

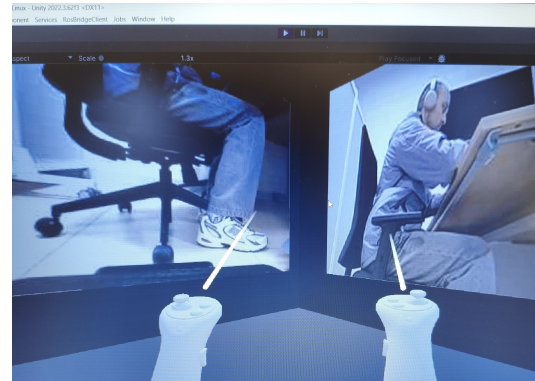


Figura 4.9: *
Telepresencia en entorno real

Figura 4.10: Perspectiva del operador en realidad virtual recibiendo los flujos de vídeo del robot (pantalla principal) y del dispositivo móvil (pantalla secundaria).

- **Recepción del vídeo del robot:** Un código de Unity recibe por UDP los *frames* JPEG enviados por el nodo Python del robot y los proyecta sobre una textura de Unity, que se muestra en la pantalla principal de la sala. Mantiene dos *buffers* independientes, uno para el *stream* RGB y otro para el de profundidad. La recepción se realiza en un hilo asíncrono para no bloquear el hilo principal. Cuando llega un *frame*, se almacena en un *buffer* compartido y en el siguiente bucle se decodifica y aplica a la textura. Solo se procesa el *frame* más reciente, descartando los intermedios. Esto garantiza latencia mínima a costa de no mostrar todos los *frames* recibidos, lo cual es el comportamiento deseado para este tipo de teleoperación en tiempo real.
- **Recepción de la cámara del interlocutor:** Se recibe por UDP los *frames* JPEG de la cámara del dispositivo Android del interlocutor y los muestra en la pantalla secundaria de la sala. El funcionamiento es análogo al anterior, recibiendo de forma asíncrona en hilo secundario, con un *buffer* de *frame* único y aplicando la textura en el hilo principal. Esto permite al operador ver la cara del interlocutor mientras teleopera el robot.
- **Envío de audio al interlocutor:** Se captura el micrófono del PC (del dispositivo por defecto de Windows) y envía el audio al dispositivo Android del interlocutor por UDP. El audio se captura a 16.000 Hz en mono y se convierte a PCM de 16 bits antes del envío. Se envían *chunks* de 320

muestras (640 bytes, equivalente a 20 ms de audio) por paquete UDP, lo que mantiene la latencia baja.

- **Recepción de audio del interlocutor:** Se escucha en un puerto UDP los paquetes de audio enviados por el dispositivo Android y se reproduce en Unity. Los paquetes se reciben en un hilo asíncrono y se encolan con un *callback*. Para evitar la acumulación de latencia por congestión de red, si la cola supera un umbral de 16.000 muestras (un segundo de audio), se vacía completamente. Esto garantiza que el audio reproducido sea siempre el más reciente, aceptando el corte momentáneo como un mal menor frente a la desincronización progresiva.

Avatar virtual y sincronización labial

El operador está representado en el entorno del interlocutor mediante un avatar virtual que se muestra en la pantalla del dispositivo Android, ya que al usar las gafas de realidad virtual no puede utilizar una webcam. Se realizó conforme a los siguientes pasos:

1. **Generación del modelo:** El modelo 3D del avatar se generó a partir de fotografías propias del operador mediante una herramienta web de creación de avatares llamada avaturn. El proceso consiste en subir imágenes del rostro desde distintos ángulos, a partir de las cuales la herramienta genera automáticamente un modelo con esos rasgos faciales, textura, esqueleto y un conjunto de *blend shapes* correspondientes a los visemas estándar. Los visemas son las posiciones de la boca y la cara asociadas a cada fonema del habla, y constituyen el mecanismo más básico sobre el que funcionan los sistemas de *lip sync* en tiempo real. El modelo se exportó en formato FBX y se importó en Unity.
2. **Sincronización labial con *uLipSync*:** La animación facial se implementó con la librería *uLipSync*, que analiza el audio del micrófono en tiempo real mediante un algoritmo, identifica el fonema más probable en cada instante y activa el *blend shape* del visema correspondiente con un peso proporcional a la probabilidad de haber acertado con el análisis. La integración con el pipeline de captura de audio se realiza a través del componente *uLipSyncAudioSource*, que recibe los datos directamente antes de su transmisión UDP desde el código de Unity encargado de procesar y enviar el audio. Esto evita capturar el micrófono dos veces.
3. **Streaming del avatar:** Una cámara de Unity enfoca directamente al avatar virtual, y un componente lo renderiza, incluyendo la animación de *lipsync*, en una *RenderTexture*, posteriormente lo codifica cada *frame* como JPEG y lo envía por UDP al dispositivo Android del interlocutor, como si fuera una cámara normal.

4.5. Aplicación Android

La aplicación Android fue desarrollada en Kotlin como una única actividad que gestiona simultáneamente los cuatro flujos de comunicación UDP independientes. La interfaz de la aplicación, que incluye la visualización del avatar y el panel de configuración de red, se muestra en la Figura 4.13.



Figura 4.11: *
Vista principal de telepresencia



Figura 4.12: *
Panel de configuración UDP

Figura 4.13: Aplicación Android actuando como nodo secundario, mostrando el avatar virtual del operador y los ajustes de conexión.

4.5.1. Recepción del vídeo del avatar

La aplicación escucha en el puerto configurado los *frames* JPEG enviados por Unity y los decodifica, actualizando un `ImageView` en la UI. El *buffer* de recepción es de 65.535 bytes, suficiente para un *frame* JPEG a la resolución y calidad moderadas.

4.5.2. Recepción del audio del operador

El audio enviado desde Unity se recibe en un *socket* UDP y se escribe directamente en un `AudioTrack` configurado a 16.000 Hz, mono, PCM de 16 bits, en modo *streaming*. El *buffer* del `AudioTrack` tiene el tamaño mínimo recomendado por el sistema para ese formato, lo que minimiza la latencia de reproducción.

4.5.3. Envío del micrófono al operador

La aplicación captura el micrófono del dispositivo mediante *AudioRecord* y envía los *chunks* de 640 bytes por UDP hacia Unity. Esto devuelve exactamente el número de bytes solicitados, lo que garantiza *chunks* de tamaño uniforme y evita acumular latencia en el *buffer* de captura.

4.5.4. Envío de la cámara al operador

La cámara trasera del dispositivo se captura descartando los *frames* anteriores y procesando solo la última imagen. Cada *frame* se convierte al formato adecuado, se comprime como JPEG y se envía por UDP a Unity. La conversión manual que se realiza es necesaria para pasar al formato NV21 que se requiere para la compresión JPEG.

4.5.5. Interfaz de configuración

La interfaz incluye campos editables para la IP de destino y los cuatro puertos de comunicación, además de mostrar la IP del propio dispositivo móvil, permitiendo reconfigurar el sistema en tiempo de ejecución sin recompilar la aplicación. Además, incorpora un botón para conectar o desconectar la conexión. Esta interfaz se puede ocultar o mostrar pulsando en la pantalla.

4.6. Entorno simulado

Para el desarrollo y las pruebas sin acceso al robot físico se utilizó el simulador Gazebo con el modelo del Turtlebot2, ejecutado dentro de una máquina virtual con Ubuntu 16.04. El entorno de simulación replica el *stack* de *software* del robot real, con *launch files* equivalentes o iguales, que se ejecutan en la máquina virtual. El PC de desarrollo interactúa de la misma manera que se hace con el robot real. En la Figura 4.14 se ilustra el entorno 3D estándar proporcionado por los paquetes de simulación del Turtlebot2.

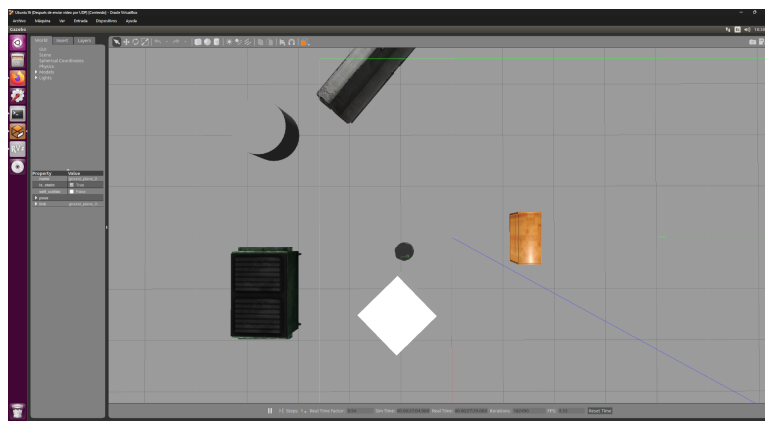


Figura 4.14: Vista cenital del entorno de pruebas simulado (*turtlebot_world*) en Gazebo.

4.6.1. Configuración de la máquina virtual

Se utilizaron dos hipervisores distintos durante el desarrollo, Hyper-V y VirtualBox, cada uno con sus propias ventajas e inconvenientes.

Hyper-V es el hipervisor integrado en Windows 11 Pro y ofrece buen rendimiento, pero presenta una limitación importante para este proyecto, ya que no tiene soporte de audio de forma nativa. Dado que parte del desarrollo implicaba probar la transmisión de audio, se optó por buscar alternativas.

VirtualBox fue la solución adoptada, principalmente por su soporte de audio y además por las *Guest Additions*, que son un conjunto de controladores y utilidades que se instalan dentro de la máquina virtual y que mejoran significativamente la experiencia y la fluidez de uso. Habilitan la aceleración gráfica 3D, permiten ajustar la resolución de pantalla dinámicamente, habilitan el portapapeles compartido entre *host* y *guest*, y mejoran el rendimiento general del sistema de entrada. Otra de las funcionalidades útiles de VirtualBox que se utilizó durante el desarrollo fue el sistema de instantáneas. Dado que el entorno obsoleto es frágil ante actualizaciones o instalaciones de paquetes que puedan romper

cosas existentes, se tomaban instantáneas del estado de la máquina virtual antes de cualquier de modificación relevante. En caso de que ocurriera un error, se podía revertir a la instantánea anterior en pocos segundos, sin necesidad de volver a reinstalar y configurar nada y sin tener que arreglar los posibles errores provocados.

Por otra parte que Unity, ejecutándose en el *host* Windows, pudiese conectarse a ROS Bridge dentro de la máquina virtual como si fuera un nodo de red independiente, se configuró el adaptador de red en modo adaptador puente. En este modo, la máquina virtual obtiene su propia dirección IP en la red local, visible desde el *host* y desde cualquier otro dispositivo en la misma red. Esto permite usar la IP de la máquina virtual directamente, exactamente igual que con el robot físico.

El entorno simulado permitió desarrollar y validar la mayor parte del proyecto, además en condiciones de red óptimas, lo que facilitó el desarrollo y la iteración de integraciones, sin depender de la disponibilidad del robot físico.

4.7. Problemas encontrados durante el desarrollo

Durante el desarrollo del sistema surgieron varios problemas técnicos que requirieron descartar funcionalidades o tomar alternativas. Se documentan a continuación los más relevantes.

4.7.1. Obsolescencia de la plataforma móvil.

Una de las limitaciones más persistentes durante todo el proyecto fue la antigüedad del *stack* de *software* del Turtlebot2. El robot ejecuta Ubuntu 16.04 con ROS Kinetic y para simularlo se utiliza Gazebo 7, versiones que alcanzaron el fin de su vida útil hace años y que ya no reciben actualizaciones ni soporte oficial. Esto tuvo varias consecuencias, como paquetes de Python incompatibles con versiones modernas, dependencias con conflictos, librerías que ya no están disponibles en los repositorios de Ubuntu 16.04, y documentación desactualizada. Esta limitación fue un gran problema durante todo el desarrollo y supuso una pérdida de tiempo considerable.

4.7.2. Acceso limitado al robot real.

Durante la mayor parte del desarrollo, las pruebas se realizaron sobre el simulador Gazebo, con acceso al robot físico de forma reiterada y constante únicamente en la fase final del proyecto. Esto provocó que varias funcionalidades que se ejecutaban correctamente en el entorno simulado fallaran al ejecutarse sobre el *hardware* real, como diferencias en los topics publicados, latencias de red no presentes en local, comportamientos del *driver* de Kobuki distintos a los del modelo simulado, configuración de la cámara Orbbec Astra errónea y problemas de sincronización. La depuración de estos errores en el robot real fue más tediosa que los problemas surgidos en el simulador.

4.7.3. Condiciones de red en el entorno real.

Las pruebas con el robot físico se realizaron en un laboratorio situado una planta sótano, con cobertura reducida. La conectividad se estableció utilizando un dispositivo móvil como punto de acceso, lo que limitaba en gran medida el ancho de banda y causaba picos de latencia. Esto afectó a la retransmisión y a la teleoperación, lo cuál obligó a reducir los parámetros de calidad y resolución respecto a los utilizados en el entorno simulado.

4.7.4. Acumulación de latencia en los búferes de datos multimedia.

La transmisión de vídeo y audio por UDP en condiciones de red inestable generaba un problema de acumulación de latencia en los búferes de recepción. Cuando la red sufría un pico de latencia, los paquetes llegaban de golpe tras la recuperación, acumulando audio y video de momentos anteriores al actual. Eso hacía que progresivamente la retransmisión se fuera retrasando respecto al presente. Para solucionarlo en el vídeo se implementó una estrategia de *frame* único, en la que cada *frame* recibido sobrescribe al anterior sin encolarse, de forma que siempre se muestra el más reciente disponible. En el caso del audio, si se acumulaba más de un segundo, se vacía la cola y se reanuda la reproducción para volver a sincronizarse con la realidad.

4.7.5. Error en el paquete RosSharp.

Durante la instalación de RosSharp 2.1.0 se encontró un error de compilación en el script `GetParamRequest.cs`. La línea problemática era `this.@default = default;` donde `default` es una palabra reservada de C# y su uso como valor

en este contexto no es válido en las versiones del compilador utilizadas por Unity. La corrección fue sustituir esa línea por `this.@default = @default;` asignando correctamente el parámetro del constructor. A pesar de lo simple de la corrección de este error, ilustra el estado de mantenimiento irregular de algunas dependencias en el ecosistema.

4.7.6. Funcionalidades descartadas.

A lo largo del desarrollo se intentaron implementar varias funcionalidades que finalmente fueron descartadas.

Entre ellas, se exploró el uso de GStreamer como tubería multimedia para la recepción de vídeo H.264 y audio en Unity. Este enfoque permitía recibir retransmisiones codificadas con mayor eficacia, lanzando un proceso externo de GStreamer desde Unity. Sin embargo, la dependencia de una instalación externa de GStreamer en el sistema, la complejidad de la gestión del proceso y los problemas de sincronización entre el GStreamer y el bucle de Unity, llevaron a descartarlo en favor de la solución UDP con JPEG, más simple y portable.

Se intentó también integrar la imagen RGB y de profundidad de la cámara del robot directamente a través de ROS Bridge, suscribiéndose al topic de imagen y recibiendo los datos como mensaje `sensor_msgs/Image`. Esto funcionaba en el simulador pero presentaba problemas de rendimiento en el robot real, donde el tamaño de los mensajes de imagen a través de ROS Bridge saturaba el *WebSocket*. Finalmente se optó por la solución ya explicada, comprimida como JPEG y enviada por UDP.

Por otra parte, se desarrolló un sistema de teleoperación alternativo, consistía en hacer clic sobre la imagen de la cámara del robot, permitiéndole al operador indicar un punto en la imagen y hacer que el robot se desplazase automáticamente hasta él. El sistema detectaba el clic sobre la textura de la cámara y obtenía sus coordenadas. También se obtenía su información de profundidad con la imagen RGB-D. Posteriormente se combinaban ambos datos junto con parámetros de la cámara para, mediante las ecuaciones de proyección inversa, obtener el punto en el espacio 3D. Una vez que se tenía ese punto, se rotaba el robot hacia el ángulo deseado y se hacía un movimiento lineal durante el tiempo estimado para recorrer la distancia. El sistema funcionaba de forma muy básica en el simulador para objetivos situados en línea relativamente recta, pero presentaba errores graves ante obstáculos o situaciones más complejas.

4.7.7. Sincronización entre hilos de ejecución.

Durante el desarrollo fue necesario gestionar la recepción simultánea de datos procedentes de distintas fuentes, como ROS Bridge y los canales UDP de audio y vídeo. Esto planteaba un problema porque Unity únicamente permite modificar la escena y sus elementos desde el hilo principal de ejecución.

Para evitar errores de concurrencia, los datos recibidos por los hilos de red no se procesaban directamente. En su lugar, se almacenaban temporalmente y se notificaba. Posteriormente, el hilo principal leía esos datos durante el bucle del juego de Unity y realizaba las modificaciones necesarias. Esto permitió desacoplar la recepción de datos de la actualización de la escena, evitando posibles bloqueos.

5

Experimentos y métricas

En este capítulo se describen las pruebas realizadas para evaluar el rendimiento del sistema desarrollado, así como las métricas empleadas para cuantificar su funcionamiento. El objetivo principal es analizar de forma objetiva aspectos clave como la latencia, el rendimiento gráfico, la calidad de la comunicación y la usabilidad general del sistema de telepresencia.

5.1. Metodología experimental

Las pruebas se realizaron en dos entornos diferentes, mostrados en la figura 5.4:

- **Entorno simulado**, el robot Turtlebot2 ejecutado en Gazebo sobre un PC conectado a una red WiFi doméstica de banda ancha. Este entorno permitió evaluar el rendimiento del sistema en condiciones de red estables y sin restricciones de ancho de banda.
- **Entorno real**, Turtlebot2 físico situado en un laboratorio en una planta sótano, con de red reducida. La conectividad se estableció mediante un dispositivo móvil actuando como punto de acceso, lo que impuso restricciones adicionales de ancho de banda y provocó variabilidad en la latencia.

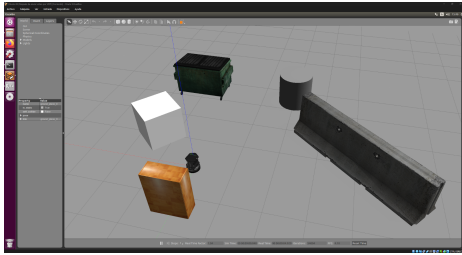


Figura 5.1: *
Entorno virtual de Gazebo



Figura 5.2: *
Entorno real puerta



Figura 5.3: *
Entorno real centro

Figura 5.4: Ambos entornos de pruebas, el simulado en Gazebo y el real en el laboratorio

En ambos entornos se realizaron las siguientes pruebas:

- Medición de latencia de red mediante *ping* continuo durante la ejecución del sistema.
- Monitorización del ancho de banda consumido desde el Turtlebot (`/sys/class/net/wlp2s0/statistics`).
- Evaluación de la tasa de refresco del vídeo transmitido y del entorno Unity.
- Pruebas de navegación libre por el entorno, completando un recorrido definido.
- Comparativa de teleoperación entre la interfaz de realidad virtual implementada y el sistema `keyboard_teleop` estándar de ROS.
- Pruebas de comunicación con interlocutores remotos mediante el sistema de audio y vídeo bidireccional.

5.2. Parámetros de configuración del sistema

La siguiente tabla recoge los parámetros utilizados durante las pruebas. Los valores conservadores de resolución y calidad de vídeo se aplicaron solo en las pruebas con el robot real, condicionados por las limitaciones de la red. En el entorno simulado se emplearon resoluciones de hasta 1920×1080 píxeles a 30 FPS sin problemas apreciables.

Parámetro	Valor
Resolución de vídeo (robot real)	320×240 px
Resolución de vídeo (simulador)	hasta 1920×1080 px
FPS vídeo (robot real)	15 FPS
FPS vídeo (simulador)	30 FPS
Calidad compresión JPEG (robot real)	30
Calidad compresión JPEG (simulador)	hasta 70
Frecuencia de audio	16.000 Hz
Formato de audio	PCM mono 16 bits
Tamaño de chunk de audio	20 ms (320 muestras / 640 bytes)
Protocolo de transmisión	UDP

Tabla 5.1: Parámetros de configuración del sistema según entorno de prueba.

5.3. Resultados

5.3.1. Latencia

En el entorno simulado con red WiFi doméstica, la latencia de red se mantuvo estable entre 2 y 5 ms, con picos puntuales de menos de 15 ms. En el entorno real (en la planta sótano con el dispositivo móvil como punto de acceso WiFi), las mediciones de *ping* también eran bajas durante la ejecución del sistema, entre 10 y 20 ms en condiciones normales, pero con picos recurrentes de 200 ms o incluso más, con pérdidas ocasionales de paquetes. Aun con estas variaciones, la latencia media durante la sesión se situó en torno a los 41 ms de media de red, lo que resultó funcional para la teleoperación.

Entorno	Latencia típica	Picos	Media
Simulado	2 - 5 ms	< 15 ms	≈ 4 ms
Real	10 - 20 ms	>200 ms (recurrentes)	≈ 41 ms

Tabla 5.2: Latencia de red medida en ambos entornos de prueba.

5.3.2. Ancho de banda

La monitorización del tráfico de red en la interfaz WiFi del Turtlebot reveló un consumo de transmisión (TX) estable de entre 100 y 130 KB/s durante el *streaming* continuo de vídeo RGB a 15 FPS y resolución 320×240. Hubo picos esporádicos de hasta 800 KB/s, probablemente coincidiendo con actualizaciones del mapa. El tráfico de recepción (RX) se mantuvo en torno a 2 KB/s, correspondiente principalmente a los comandos de teleoperación. En el entorno simulado, con mayor ancho de banda disponible, fue posible operar a 1080p y 30 FPS sin

saturar la red y sin picos de latencia. En el entorno simulado, el tráfico TX se situó entre 3,4 y 5,1 MB/s al transmitir vídeo a 1080p y 30 FPS, mientras que el RX permaneció por debajo de 8 KB/s.

Entorno	TX (estable)	Picos TX	RX
Real	100 - 130 KB/s	800 KB/s	≈ 2 KB/s
Simulado	3,4 - 5,1 MB/s	–	< 8 KB/s

Tabla 5.3: Consumo de ancho de banda en la interfaz WiFi del Turtlebot según el entorno.

5.3.3. Tasa de refresco

El entorno de Unity se mantuvo una tasa de refresco superior a 90 FPS en ambos entornos de prueba, independientemente de la calidad de la transmisión de vídeo, aunque en el PC de desarrollo, al ser mucho más potente que el portátil utilizado en el entorno de pruebas real, conseguía alcanzar FPS bastante mayores. La recepción de la retransmisión de vídeo del robot funcionaba a 15 FPS en el entorno real y a 30 FPS en el simulado, lo cual fue suficiente para la teleoperación aunque con una fluidez visual inferior en el caso real.

5.3.4. Calidad audiovisual

El audio se transmitió de forma bidireccional a 16.000 Hz, mono, PCM de 16 bits, en *chunks* de 20 ms (640 bytes por paquete UDP). La comunicación de voz resultó inteligible en ambos entornos. El vídeo en el entorno real operó a resolución reducida por las limitaciones de red descritas, lo que provocó una imagen visiblemente pixelada aunque funcional para la teleoperación. En el entorno simulado, la transmisión a 1080p con alta calidad producía una imagen nítida comparable a una videollamada estándar.

5.3.5. Tareas de navegación

En las pruebas de navegación, el robot completó de forma consistente recorridos a través de obstáculos por el área de prueba tanto en el simulador como en el espacio real, alrededor del laboratorio, en los mismos entornos que se muestran en la figura 5.4. El control mediante VR resultó efectivo. Al compararlo con `keyboard_teleop`, para cualquier operador con experiencia previa en videojuegos, la preferencia entre ambos controles se reduce a una elección personal.

6

Conclusiones y trabajos futuros

En este capítulo final se presenta una reflexión global del trabajo realizado, analizando el grado de cumplimiento de los objetivos planteados, los resultados obtenidos y las principales aportaciones del sistema desarrollado. Asimismo, se exponen las limitaciones identificadas durante el desarrollo del proyecto y se plantean posibles líneas de trabajo futuras.

6.1. Discusión de resultados

Los resultados obtenidos nos dan conclusiones diferenciadas según el entorno de prueba. En condiciones de red óptimas, el sistema demostró ser capaz de operar con parámetros de calidad altos, baja latencia y audio inteligible. Esto confirma que las limitaciones con el robot real son exclusivas de las limitaciones de la infraestructura de la red.

En el entorno real, la combinación de cobertura reducida por la localización del entorno de pruebas y el uso de un dispositivo móvil como punto de acceso WiFi, introdujo picos en la latencia y un ancho de banda limitado, obligando a operar con resoluciones y tasas de refresco reducidas. Aun así, el sistema seguía siendo funcional y la comunicación de voz funcionaba sin interrupciones, lo que demuestra que el sistema y las soluciones como la reconexión automática, el descarte de *frames* viejos y la transmisión sin acumular *buffer*, funcionan de forma robusta.

La tasa de refresco de Unity siempre por encima de 90 FPS lo hace óptimo para realidad virtual en ambos entornos de prueba. Esto demuestra que el sistema funciona, y que mejorando la infraestructura de red, se podrían obtener calidades altas en el entorno real sin realizar cambios en el proyecto.

6.2. Cumplimiento de objetivos

A continuación, se analiza el grado de cumplimiento de los objetivos definidos al inicio del trabajo.

6.2.1. Objetivo general

El objetivo general del proyecto, consistente en el desarrollo de un sistema de telepresencia robótica basado en realidad virtual que permita la interacción remota con un entorno real, ha sido alcanzado satisfactoriamente. El sistema implementado permite tanto la teleoperación del robot como la comunicación multimodal en tiempo real, reproduciendo de forma razonablemente natural una interacción presencial. La integración de ROS, Unity y dispositivos de realidad virtual ha permitido desarrollar una plataforma funcional, inmersiva y adaptable, capaz de ejecutarse sobre *hardware* de coste moderado manteniendo un rendimiento adecuado para aplicaciones de telepresencia.

6.2.2. Objetivos específicos

- **Diseño de interfaz en realidad virtual:** se ha desarrollado una interfaz accesible, clara e intuitiva, permitiendo al usuario comprender rápidamente el funcionamiento del sistema y utilizarlo sin necesidad de un aprendizaje complejo.
- **Teleoperación del robot:** se ha implementado con éxito el control del Turtlebot 2 desde el entorno de realidad virtual, logrando una integración efectiva entre ROS y Unity que permite traducir las acciones del usuario en comandos de movimiento del robot en tiempo real.
- **Comunicación multimodal:** se ha desarrollado un sistema completo de transmisión bidireccional de audio y vídeo, incluyendo la transmisión de vídeo desde el robot hacia el entorno de realidad virtual, una fuente adicional desde un dispositivo móvil y el envío de vídeo desde Unity hacia el dispositivo móvil mediante un avatar virtual con sincronización labial. Todo ello permite mantener una interacción más natural entre los usuarios remotos.

- **Análisis de usabilidad y rendimiento:** se han realizado pruebas orientadas a evaluar la latencia, el rendimiento gráfico, la estabilidad de la comunicación y la experiencia de usuario, confirmando la viabilidad del sistema para escenarios reales de telepresencia.
- **Modularidad y escalabilidad:** el sistema ha sido desarrollado siguiendo una arquitectura modular que facilita la integración de nuevos dispositivos, sensores y funcionalidades. Actualmente, la solución es compatible con gran parte de dispositivos HMD y con la inmensa mayoría de dispositivos Android (aproximadamente el 98.4%) y permite futuras ampliaciones sin necesidad de rediseñar completamente el sistema.

6.3. Conclusiones generales

El desarrollo de este trabajo ha permitido validar la viabilidad de utilizar tecnologías de realidad virtual y robótica móvil para la creación de un sistema de telepresencia inmersiva funcional y de bajo coste relativo. La integración entre ROS, Unity y dispositivos de realidad virtual ha demostrado ser una solución adecuada para implementar entornos de teleoperación capaces de proporcionar interacción remota en tiempo real de forma intuitiva y accesible.

El sistema desarrollado permite no solo el control remoto del Turtlebot2 desde un entorno de realidad virtual, sino también el establecimiento de una comunicación multimodal basada en transmisión bidireccional de audio y vídeo, complementada mediante la representación virtual del operador a través de un avatar con sincronización labial. Todo ello contribuye a generar una experiencia de telepresencia más natural y cercana que la obtenida mediante interfaces tradicionales de teleoperación.

Asimismo, el proyecto pone de manifiesto el potencial de las tecnologías inmersivas aplicadas a la robótica de servicio y telepresencia, especialmente en ámbitos donde la interacción remota resulta relevante, como la asistencia, la supervisión remota, la educación o la comunicación a distancia. La arquitectura modular implementada facilita además la incorporación de futuras mejoras y la adaptación del sistema a nuevos dispositivos y escenarios de uso.

Aunque existen limitaciones relacionadas principalmente con la dependencia de la calidad de red, el ancho de banda disponible y las restricciones *hardware* de la plataforma utilizada, los resultados obtenidos muestran que es posible alcanzar un rendimiento adecuado para aplicaciones de telepresencia en tiempo real utilizando herramientas y dispositivos ampliamente accesibles.

En conjunto, el trabajo realizado constituye una base sólida para futuras ampliaciones y demuestra las posibilidades que ofrece la combinación de robótica,

realidad virtual y comunicación multimedia en el desarrollo de sistemas de interacción remota más inmersivos, naturales y versátiles.

6.4. Trabajos futuros y posibles mejoras

A partir del proyecto desarrollado existen multitud de mejoras y expansiones que permitirían aumentar las funcionalidades del sistema y la calidad del mismo:

- **Incorporar una cámara estereoscópica o una cámara omnidireccional**, o idealmente la combinación de ambas, permitiría mejorar significativamente la percepción del entorno; la cámara estereoscópica aportaría información de profundidad para estimar distancias con mayor precisión, mientras que la omnidireccional ofrecería una visión global en 360 grados, aumentando la conciencia espacial del usuario y permitiendo una teleoperación más natural e inmersiva, sin necesidad de redirigir el robot para explorar alrededor, mucho más práctico que el sistema actual basado en una cámara RGB-D.
- **Incorporar un brazo robótico** al Turtlebot permitiría no solo la telepresencia visual, sino también la interacción física con el entorno, ampliando significativamente las aplicaciones del sistema hacia tareas como manipulación de objetos o asistencia remota.
- **Mejorar la calidad de la comunicación multimodal** en lo referente al audio y vídeo en tiempo real, mediante el uso de protocolos más robustos y optimizados, permitiría una experiencia más fluida y cercana a sistemas de telepresencia y videollamada profesionales, reduciendo latencias y mejorando la estabilidad de la conexión.
- **Implementar sistemas de retroalimentación háptica** aprovechando los sensores ya disponibles en el Turtlebot2, como los sensores de choque, sensores de acantilado o detección de caída de rueda, permitiría transmitir al usuario información física del entorno mediante vibraciones en un chaleco háptico u otros estímulos, aumentando la percepción de inmersión y seguridad durante la teleoperación.
- **Añadir un control mediante seguimiento de manos** para una interacción más natural e intuitiva, eliminando la dependencia de dispositivos físicos y acercando la experiencia a un control más directo del robot mediante gestos.
- **Incorporar un modo de seguimiento autónomo** permitiría que el Turtlebot2 fuese capaz de seguir a una persona de forma automática dentro de un entorno sin necesidad de control directo por parte del operador, para

que, al facilitarle dicha tarea, pueda centrarse en la comunicación. Esto resultaría especialmente útil en aplicaciones como guiado, asistencia remota o acompañamiento.

- **Actualizar la plataforma robótica** a un modelo más reciente, como el Turtlebot4 u otros equivalentes, permitiría aprovechar mejoras tanto a nivel *hardware* como sobre todo a nivel de *software*, incluyendo mayor capacidad de procesamiento, mejores sensores y compatibilidad con versiones más modernas de ROS; esto facilitaría la integración de nuevas funcionalidades, el uso de librerías actuales y una mayor mantenibilidad del sistema a largo plazo.

Estas mejoras abrirían la puerta a un sistema más versátil y aplicable a una mayor variedad de escenarios reales, permitiendo una interacción más natural y enriquecida entre humano y robot en contextos de telepresencia avanzada.

Bibliografía

- [1] Siemens AG. *ROS# Documentation*, 2024. URL <https://github.com/siemens/ros-sharp>.
- [2] Jose Berengueres. Using smartphones for low-cost robotics. *arXiv preprint arXiv:1807.06003*, 2018. doi: 10.48550/arXiv.1807.06003. URL <https://arxiv.org/abs/1807.06003>.
- [3] Jean Botev and Francisco J. Rodríguez Lera. Immersive robotic telepresence for remote educational scenarios. *Sustainability*, 13(9):4717, 2021. doi: 10.3390/sul3094717. URL https://www.researchgate.net/publication/368005139_Immersive_Robotic_Telepresence_for_Remote_Educational_Scenarios.
- [4] Robert Frank Codd-Downey, Parisa Mojiri Forooshani, Andrew Speers, Hui Wang, and Michael Jenkin. From ros to unity: Leveraging robot and virtual environment middleware for immersive teleoperation. In *2014 IEEE International Conference on Information and Automation (ICIA)*, pages 932–936, 2014. doi: 10.1109/ICInfA.2014.6932785. URL https://www.researchgate.net/publication/286716293_From_ROS_to_unity_Leveraging_robot_and_virtual_environment_middleware_for_immersive_teleoperation.
- [5] Christyan Cruz Ulloa, David Domínguez, Jaime del Cerro, and Antonio Barrientos. Analysis of MR-VR tele-operation methods for legged-manipulator robots. *Virtual Reality*, 28:131, 2024. doi: 10.1007/s10055-024-01021-z. URL <https://doi.org/10.1007/s10055-024-01021-z>.
- [6] Anthony H. Dugan, Rustem Sadykov, Daniel Roozbahani, Marjan Alizadeh, and Heikki Handroos. Immersive telepresence via a humanoid robotic head using a vr-headset with real-time stereoscopic vision and binaural audio. *Journal of Intelligent & Robotic Systems*, 111(2), 2025. doi: 10.1007/s10846-025-02254-z. URL https://www.researchgate.net/publication/391400390_Immersive_Telepresence_via_a_Humanoid_Robotic_Head_Using_a_VR-Headset_with_Real-Time_Stereoscopic_Vision_and_Binaural_Audio.
- [7] R. C. Goertz. Master-slave manipulator. Technical report, Argonne National Laboratory, Argonne, Illinois, March 1949. URL <https://digital.library.unt.edu/ark:/67531/metadc828171/>. Accessed: 2026-05-28.

- [8] Matheus Nicolás Hernández, Brenno Domingues, Roberto Simoni, Douglas Negri, Diego De Souza, and Luís Gonzaga Trabasso. Robot teleoperation via virtual reality: A unity and ros approach. In *2024 3rd International Conference on Automation, Robotics and Computer Engineering (ICARCE)*, 2024. doi: 10.1109/ICARCE63054.2024.00012. URL https://www.researchgate.net/publication/391389407_Robot_Teleoperation_via_Virtual_Reality_A_Unity_and_ROS_Approach.
- [9] Fernando Agustín Hernández Gobertti. Immersive teleoperation of a robotic arm with grip recognition and haptic feedback. Master's thesis, Universidad Politécnica de Valencia, 2024. URL <https://riunet.upv.es/handle/10251/209550>.
- [10] Rebecca Hetrick, Nicholas Amerson, Boyoung Kim, Eric Rosen, Ewart de Visser, and Elizabeth K. Phillips. Comparing virtual reality interfaces for the teleoperation of robots. In *2020 Systems and Information Engineering Design Symposium (SIEDS)*, 2020. doi: 10.1109/SIEDS49339.2020.9106630. URL https://www.researchgate.net/publication/341842633_Comparing_Virtual_Reality_Interfaces_for_the_Teleoperation_of_Robots.
- [11] Phil Holdorf. Automated generation of source files for a ros-unity interface. Bachelor thesis, University of Hamburg, TAMS, 2019. URL https://tams.informatik.uni-hamburg.de/publications/2019/Bachelorarbeit_Holdorf.pdf.
- [12] Ke Li, Reinhard Bacher, Susanne Schmidt, Wim Leemans, and Frank Steinicke. Reality fusion: Robust real-time immersive mobile robot teleoperation with volumetric visual data fusion. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024. URL <https://www.edit.fis.uni-hamburg.de/ws/files/55138101>. Github: url = <https://github.com/uhhhci/RealityFusion>.
- [13] Jacques Marescaux, Joel Leroy, Michel Gagner, Francesco Rubino, Didier Mutter, Michel Vix, Steven E. Butner, and Michelle K. Smith. Transatlantic robot-assisted telesurgery. *Nature*, 413:379–380, 2001. doi: 10.1038/35096636. URL <https://www.nature.com/articles/35096636>.
- [14] Francisco Melendez-Fernandez, Cipriano Galindo, and Javier Gonzalez-Jimenez. A web-based solution for robotic telepresence. *International Journal of Advanced Robotic Systems*, 2017. doi: 10.1177/1729881417743738. URL <https://journals.sagepub.com/doi/10.1177/1729881417743738>.
- [15] Katherine J. Mimnaugh, Markku Suomalainen, Israel Becerra, Eliezer Lozano, Rafael Murrieta-Cid, and Steven M. LaValle. Analysis of User Preferences for Robot Motions in Immersive Telepresence. *arXiv preprint arXiv:2103.03496*, 2021. URL <https://arxiv.org/abs/2103.03496>.
- [16] José Carlos Penalva Maciá. Teleoperación de un robot mediante reconocimiento de gestos. Trabajo fin de grado, Universidad de Alicante, 2024. URL https://rua.ua.es/bitstream/10045/144243/1/Teleoperacion_de_un_robot_mediante_reconocimiento__Penalva_Macia_Jose_Carlos.pdf.

-
- [17] Morgan Quigley et al. Ros: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009. URL <https://www.ros.org/>.
 - [18] Open Robotics. *TurtleBot 2 Documentation*, 2016. URL <https://wiki.ros.org/Robots/TurtleBot>.
 - [19] Open Robotics. *Gazebo Simulator Documentation*, 2024. URL <https://gazebo.org/home>.
 - [20] Max Schwarz, Christian Lenz, Andre Rochow, Michael Schreiber, and Sven Behnke. NimbRo Avatar: Interactive Immersive Telepresence with Force-Feedback Telemanipulation. *arXiv preprint arXiv:2109.13772*, 2021. URL <https://arxiv.org/abs/2109.13772>.
 - [21] Stanislav Svediroh and Ludek Zalud. But telepresence: A fully open-source vr framework for low-latency stereoscopic video streaming with end-to-end latency instrumentation. *Research Square*, 2026. doi: 10.21203/rs.3.rs-8855703/v1. URL https://www.researchgate.net/publication/400888224_BUT_Telepresence_A_Fully_Open-Source_VR_Framework_for_Low-Latency_Stereoscopic_Video_Streaming_with_End-to-End_Latency_Instrumentation.
 - [22] Unity Technologies. *Unity Robotics Hub*, 2024. URL <https://github.com/Unity-Technologies/Unity-Robotics-Hub>.
 - [23] Robot Web Tools. *ROSBridge Suite Documentation*, 2024. URL https://wiki.ros.org/rosbridge_suite.
 - [24] Manuel Tovar Martínez. Interfaz inmersiva de realidad virtual para la teleoperación de robots móviles. Master’s thesis, Universidad de Alicante, 2021. URL <https://rua.ua.es/entities/publication/289de83b-eacb-4437-b62a-ae04893b07f0>. Trabajo Fin de Grado.
 - [25] David Whitney, Eric Rosen, Elizabeth Phillips, George Konidaris, and Stefanie Tellex. Comparing robot grasping teleoperation across desktop and virtual reality with ros reality. In *International Symposium on Robotics Research (ISRR)*, 2018. URL https://cs.brown.edu/people/gdk/pubs/vr_teleop.pdf. Github: url = https://github.com/h2r/ros_reality.
 - [26] David Whitney, Eric Rosen, Daniel Ullman, Elizabeth Phillips, and Stefanie Tellex. Ros reality: A virtual reality framework using consumer-grade hardware for ros-enabled robots. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018. URL https://www.researchgate.net/publication/326693689_ROS_Reality_A_Virtual_Reality_Framework_Using_Consumer-Grade_Hardware_for_ROS-Enabled_Robots.
 - [27] Adhi Widagdo, Markku Suomalainen, Basak Sakcak, Katherine J. Mimnaugh, Juho Kalliokoski, Alexis P. Chambers, Timo Ojala, and Steven M. LaValle. A Study of Preference and Comfort for Users Immersed in a Telepresence Robot.

BIBLIOGRAFÍA

- arXiv preprint arXiv:2203.02699*, 2022. URL <https://arxiv.org/abs/2203.02699>.
- [28] Karim Youssef, Sherif Said, Samer Al Kork, and Taha Beyrouthy. Telepresence in the recent literature with a focus on robotic platforms, applications and challenges. *Robotics*, 12(4):111, 2023. doi: 10.3390/robotics12040111. URL <https://www.mdpi.com/2218-6581/12/4/111>.